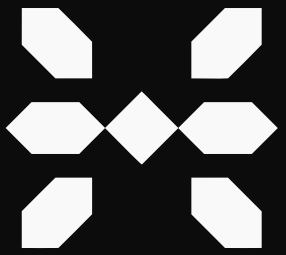
HOOX · PYNE · DOCUMENTATION



RUNTIME ENGINEERS · STRATEGY AUTHORS

RUNTIME MANUAL

Evaluator, series model, builtins, strategy, compiler

Open-source Pine Script evaluation framework

2026-11-01 DIN A4 · HTTPS://HOOX.SH/PYNE/DOCS OPEN SOURCE

CONTENTS

01 Overview

02 Series And History

03 Expressions Statements

04 Overview

05 Technical

06 Strategy

07 Collections

08 Drawing Plotting

09 Request Input

10 Libraries

11 Events

12 Alerts

13 Overview

14 Numba

15 Strategy Broker

16 Parity

OVERVIEW

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Runtime" description: "Bar-loop evaluator — series caps, incremental TA, alerts, fill export, foreign request.security → na, and interpret↔compile parity."

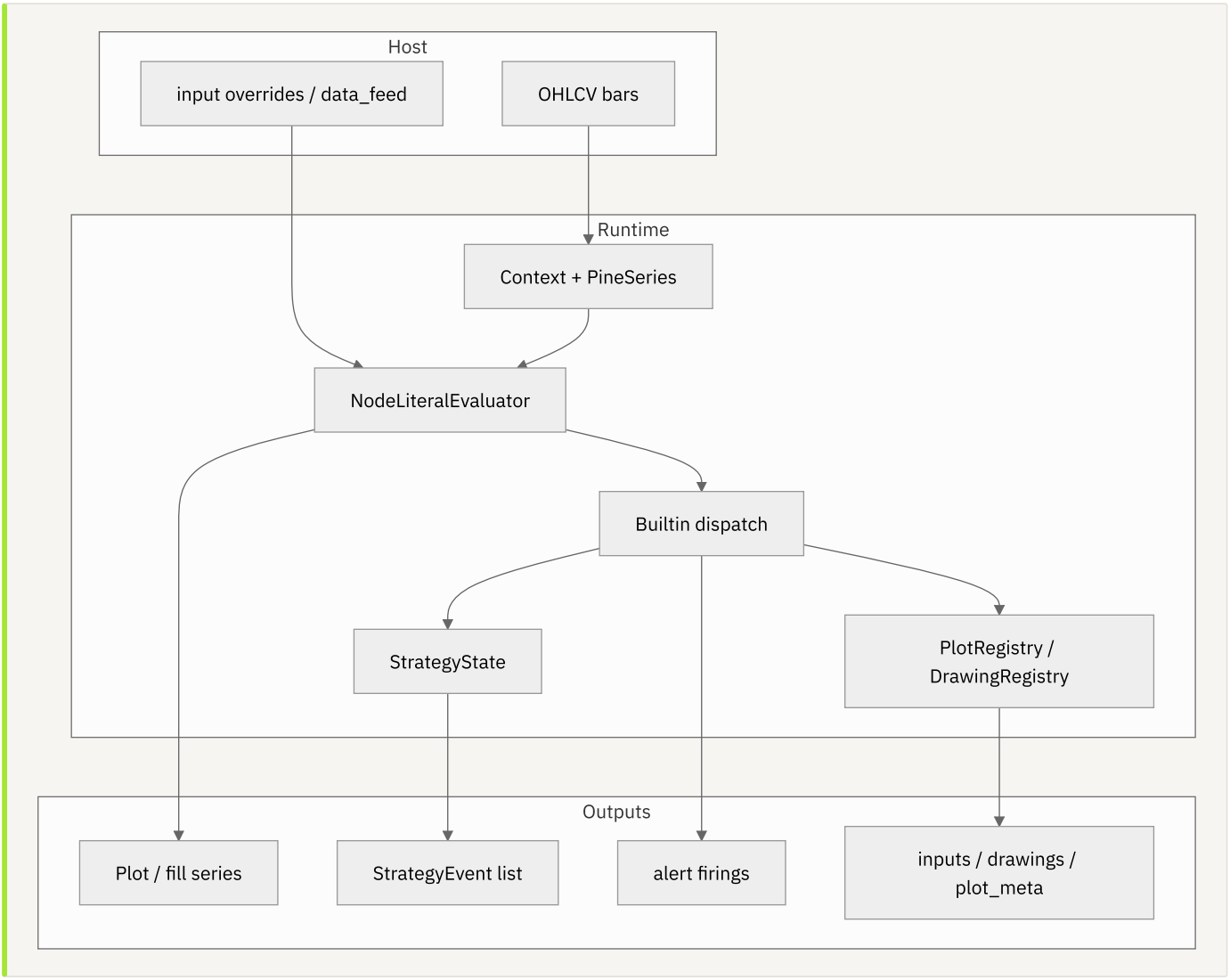
RUNTIME

Abstract

PYNE's runtime is the deterministic **bar-loop** that turns a parsed ASDL AST into series values, drawing side-effects, strategy events, and **alerts**. It is deliberately host-agnostic: the same visitor mixins power the library API, the Flask Pro API (`Runtime.run`), browser Pyodide, and edge workers. A second path—source-to-source compilation into a Numba or object-mode bar loop—trades interpretive flexibility for throughput on long histories; Pro API defaults to `mode=auto`.

This section documents **semantics**, not the chart. AXIS consumes plots / `fill` / drawings; hosts export alerts (optional L2 webhooks); HOOX consumes strategy events. None of those hosts is required to evaluate a script.

Conceptual model



Execution modes share the same parse tree (Pro API / `Runtime` also expose `auto`):

Mode	Entry	Bar loop	Typical use
Interpret	`NodeLiteralEvaluator.visit(tree)` per bar	Host updates `PineSeries` / context, re-visits AST	Full language surface, strategy, drawings, alerts, `request.*`
Compile	`pynescript.compiler.engine.compile_script`	Generated `for __bar_idx in range(n)` over OHLCV + **`time_arr`**	Numeric indicators (Numba) or object-mode subset
Auto	`Runtime.run(..., mode="auto")`	Compile when safe; interpret on fallback	Pro API default — prefer throughput without hard-failing exotic scripts

Compile inputs are contiguous arrays: `open_arr` … `vol_arr` plus **`time_arr`** (bar-open Unix ms; synthetic `bar_index * 60_000` when the host omits times) so `time` / calendar builtins match the interpret `time` series. Disk IR cache + product warm-compile (prewarm API/CLI) cut cold JIT latency. See Compiler overview and Interpret ↔ compile parity.

`request.security` (landed): interpret resamples same-symbol simple OHLCV and allowlisted `ta.sma` / `ema` / `rsi` / `atr` on a **coarser** TF (last **completed** HTF bucket only). Compile passthrough is same-symbol simple OHLCV only. Foreign symbols and complex / nested expressions resolve to **`na`** on both hosts (no inventing chart close as foreign fundamentals). Policy tags land on `result["meta"]["request_security"]`. Real multi-symbol feeds remain adapter work (roadmap **B1**).

Series caps / incremental TA: host list trim defaults **on** (`PYNE_SERIES_CAP`); bar-mode TA hot path defaults **on** (`PYNE_TA_INCREMENTAL`). Drawing GC honors `max_*_count` on the interpret registry; `fill` exports series keys + `plot_meta` for AXIS bands.

Interpret hot path (0.3.10 / Round 9): the AST walker inlines Assign / `Expr(Call)` after the first bar (function/type/import decls stay locked), skips unused derived OHLCV series (`hl2` / `hlc3` / `ohlc4` / `tr` unless named, `input.source`, or `ta.vwap`), and reuses plot cells by call-site. Same-machine bench @ 2000 bars vs 0.3.9: minimal **2.7×**, `ta.sma` **2.0×**, `ta_combo` **1.45×**. `PYNE_SERIES_RING` stays **off**.

Host flags and knobs

Knob	Default	Effect
`PYNE_SERIES_CAP`	**on**	Trim chronological `current_series` lists (`max_bars_back` / 256)
`PYNE_TA_INCREMENTAL`	**on**	Call-site incremental `ta.*` in bar mode
`PYNE_SERIES_RING`	**off**	Chronological ring lookback; skip dual list write when on
`PYNE_LIGHT_PLOTS`	**off**	Skip plot columns + input meta (corpus OK/fail only)
`PYNE_RUNTIME_MODE`	`interpret`	Default when `Runtime.run(mode=)` is omitted (`auto` is the Pro API body default)
`timeout_seconds=`	`None`	Interpret wall-clock budget; checked every 32 bars → `timed_out` + `error_kind=runtime`
`libraries=`	`[]`	`{namespace, name, version, source}` registered before `import` (auto forwards into interpret fallback)
`realtime_*`	historical	Interpret-only `varip` / forming-bar simulation (`realtime_last_bar` / `ticks` / `bars` / `from_bar`)
`Runtime.create_session`	interpret	Persistent `InterpretSession` (0.6.1): delta `append_bars`, forming-bar `update_last_bar`; compile has no resumable state

Interface surface

Concern	Start here
Series history, `[]`, `var` / `varip`, `PYNE_SERIES_CAP`, `na`	Series & history
Expressions, statements, control flow	Expressions & statements
Builtin namespaces (`ta.*`, `strategy.*`, ...)	Builtins hub
Drawing / `plot` / `fill` / `max_*_count` GC	Drawing & plotting
`request.*` / `input.*` (foreign → `na`)	request / input
Library `export` / `import`	Libraries
`StrategyEvent` parity contract	Events
`alert()` / webhooks	Alerts
Numba / object-mode compiler	Compiler overview
Interpret vs compile plot parity	Parity testing
Incremental interpret sessions (`create_session`)	Evaluate scripts

Library callers typically use `NodeLiteralEvaluator` or the package **`pynescript.runtime.Runtime`** (Pro API re-export) rather than wiring the bar loop by hand:

```
from pynescript.ast.evaluator import NodeLiteralEvaluator
from pynescript.runtime import Runtime  # package SoT (0.3.4+)

ev = NodeLiteralEvaluator(context={"close": [1.0, 2.0, 3.0], "bar_index": 2})
result = ev.evaluate_script('//@version=6\nindicator("x")\nplot(close)')

# Full bar loop (preferred host API):
# Runtime(symbol="CHART").run(
#     source, ohlcv_data=bars, mode="auto",
#     timeout_seconds=None, libraries=None,
# )
# Incremental interpret (0.6.1):
# sess = Runtime(symbol="CHART").create_session(source, bars)
# sess.append_bars(new_closed_bars)
# sess.update_last_bar(forming_bar)
```

Hosts that own OHLCV (Pro API, workers) use the package host: push bar → fill pending orders → `visit(tree)` → drain events → collect plots. **SoT:** `src/pynescript/runtime/` (`host.py`, `series.py`, `evaluator.py`). `backend.runtime` / `backend.series` / `backend.evaluator` are identity re-export shims for the Pro API.

Internals

Path	Role
`src/pynescript/runtime/`	Package Runtime SoT — host, series, CustomEvaluator
`src/pynescript/ast/evaluator/`	Mixin evaluators (base, names, expressions, statements, libraries, events)
`src/pynescript/ast/evaluator/builtins/`	Builtin dispatch tables
`src/pynescript/compiler/`	`CompilerVisitor`, Numba builtins, strategy broker, engine façade
`backend/runtime.py`	Compat re-export (`sys.modules` alias) of `pynescript.runtime.host`
`tests/test_evaluator.py`, `tests/test_parity.py`, `tests/test_strategy_*.py`, `tests/test_first_party_ta_goldens.py`	Semantic oracles + dual-host TA goldens

Composition of the full interpreter:

```
BaseEvaluator
  + LiteralEvaluator
  + NameEvaluator
  + ExpressionEvaluator
  + StatementEvaluator
  + BuiltinEvaluator
  → NodeLiteralEvaluator
```

`BuiltinEvaluator` itself aggregates mixins (`TechnicalAnalysisMixin`, `StrategyBuiltinsMixin`, `PlottingFunctionsMixin`, …) into one dispatch map built at construction time.

Invariants & edge cases

1. **Bar re-entrancy.** The AST is visited once per bar. Function/type definitions lock after bar 0 (`_pine_defs_locked`) so multi-dispatch tables do not grow (O(\text{bars}^2)).
2. **Order of broker steps.** Pending limit/stop fills run **before** script body evaluation on each bar (interpreter and compile object-mode agree). Pending-fill averaging when pyramiding ≤ 0 matches both brokers (**F2**).
3. **`na` is `None`.** Unresolved missing data, OOB history, and arithmetic with missing operands produce Python `None`, not IEEE NaN—except on the Numba path, which uses `np.nan` as the array sentinel. Cross-mode plot compares must normalize (`scripts/compare_interp_compile.py`).
4. **Series caps.** Unbounded `current_series` growth is capped (`PYNE_SERIES_CAP` default ON, `max_bars_back` / `_SERIES_MAX`) so long histories stay memory-bounded (**T1**).
5. **Incremental TA.** Hot-path kernels (MAs, oscillators, `bb` / `kama` / `cmo` / `stochrsi`, volume `obv` / `wad` / `cmf` / `klinger` / `nvi` / `pvi`, …) use call-site incremental updates in bar mode (`PYNE_TA_INCREMENTAL=0` to disable).
6. **Strategy state is per-evaluator.** `StrategyState` lives on the instance (`evaluator._strategy_state`), never as a process-global singleton—required for concurrent runs and parity tests.
7. **Drawing/plot registries are side channels.** Visual objects do not participate in pure expression values except where `plot()` returns a plot id for `fill()`. Hosts export `fill` bands for AXIS via series + `plot_meta` (and

`__drawings` on compile). Drawing objects honor `max_*_count` GC (package + Pro API + AXIS Pyodide).

8. **Alerts are a host side-channel.** `alert()` / `alertcondition()` firings are not strategy events; dual-host export + L2 webhooks are documented under Alerts.
9. **Host series are not aliased on assign.** `last = time` (or `open` / `high` /…) copies the current scalar into a fresh series for `last` —see Series & history.
10. **Omitted bid/ask stay `na`.** Host `bid` / `ask` update only when those keys appear on a bar dict; they are not synthesized from close.

Worked example — minimal bar loop mental model

```
for bar_index, bar in ohlcv:
    open/high/low/close.update(bar)
    context[bar_index, time, barstate, …] = …
    process_pending_orders(OHLC)   # broker sim
    evaluator.visit(ast)           # script body
    events += drain_events()
    series.append(plot values)
```

Compile mode collapses this into one generated function over full arrays (`open_arr` … `vol_arr`, `time_arr`), with `__bar_idx` standing in for `bar_index`.

Failure modes

Symptom	Likely cause
`unexpected type of node: …`	AST node not implemented in any mixin's `visit_*`
`Unknown built-in function: …`	Missing dispatch key or wrong qualified-name resolution
Divergent strategy events vs TV	Partial-fill / OCA / risk settings, or fill timing vs bar
Numba path raises / falls back	Script uses UDT/map/drawing → object mode; or Numba not installed
Plots empty but no error	Host forgot to collect `plot_outputs` / `PlotRegistry` after visit; or `PYNE_LIGHT_PLOTS=1`
`timed_out` + partial series	`timeout_seconds` elapsed (checked every 32 interpret bars)

See also

- Language core — grammar, AST, types before evaluation
- Alerts — `alert()` engine + L2 webhooks
- Pro API runtime bridge
- Interpret ↔ compile parity — `scripts/compare_interp_compile.py`
- Numerical validation
- Roadmap — H1/H2/T1/T2/F1/F2/L2/P1p official builtins landed; optional set0x unmeasured
- pyne-worker
- AXIS docs — optional AXIS for series/drawings

SERIES AND HISTORY

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

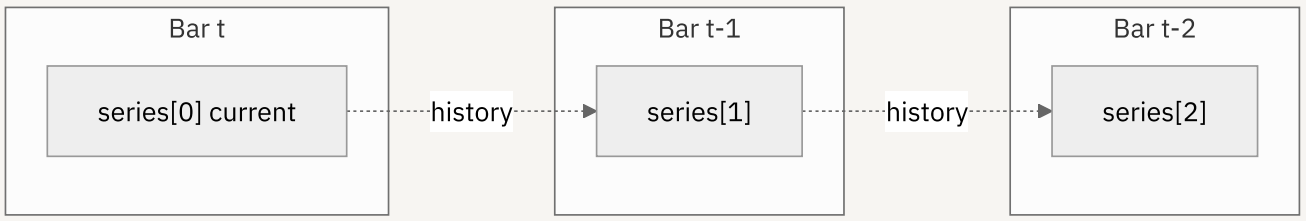
title: "Series and history" description: "Pine series model: history operator, na propagation, var/varip persistence, and bar-mode scalars."

SERIES AND HISTORY

Abstract

Pine's core data structure is the **series**: a value that evolves bar by bar, with random access to past values via the history operator `[]`. PYNE implements that model with host-managed series wrappers (`PineSeries` in the Pro API runtime), plain Python lists in unit tests, and flat `numpy` arrays on the compile path. Understanding history indexing and `na` is prerequisite to every builtin and strategy claim.

Conceptual model



Pine indices are **most-recent-first**:

Pine	Meaning	List representation (chronological)
`close[0]`	Current bar	`list[-1]`
`close[1]`	Previous bar	`list[-2]`
`close[n]` with (n \ge \text{len})	Out of range	`na` (`None`)

Negative offsets are **not** Python wraparound; they **soft-fail** to `na` (`None`).

Interface surface

History operator (`visit_Subscript`)

Implemented in `NameEvaluator.visit_Subscript` :

- Float indices coerce to `int` (e.g. `depth / 2`); NaN index → `na` .
- Lists map Pine index (i) to Python index `-(i + 1)` .
- Scalars: `x[0]` is `x` ; `x[i]` for (i > 0) is `na` (no history buffer).
- Matrices use `[row, col]` (tuple/list of length 2), not series semantics.

Series wrappers

Hosts expose OHLCV as objects with:

- `.current` — scalar for the active bar
- `.history` — most-recent-first buffer (deque or list)

Arithmetic in `ExpressionEvaluator` coerces such wrappers via `_as_scalar_operand` so `close + 1` does not attempt object addition.

Assigning host series (copy, not alias)

Binding a **host** multi-bar series into a **new** name copies the current scalar into a fresh series for that name—it does **not** alias the underlying `open` / `high` / `low` / `close` / `volume` / `time` buffers:

```
last = time          // copy of current time into series `last`
last := last + 1     // mutates only `last`, never the host `time` buffer
```

Implemented in `StatementEvaluator._bind_series_name` : if the RHS has `.current` / `.history` / `.update` (a host or tracked series), the RHS is reduced to its **current scalar** before allocating or updating the LHS series. Same-bar reassignment (`x = 0` then `x := expr`) overwrites the current sample so `x[1]` remains the previous bar's

final value. This prevents scripts such as dividend TTM time tracking from corrupting `time[j]` history when they rebind through a local.

UDF series parameters (0.6.2)

A Pine function parameter that is subscripted (`s[1]` inside `f(s) => …`) is a **series of that argument**, not a scalar that happens to be named `s` . Interpret now allocates a per-call-site history wrapper when the caller passed a scalar or a host-series current (`src = close; updown(src)`). Compile already tracked UDF series state; the two hosts agree on official TradingView® builtins (P1p **0 MISMATCH**). Nested calls get distinct streams. See Expressions & statements.

`var` / `varip` / `const`

Qualifier	Semantics in PYNE
`var` / `varip`	Initializer runs on first **execution** of that declaration site (tracked in `_var_declarations`), not only `bar_index == 0` . Later bars skip re-init so the value carries. History-tracked `var` series get a **start-of-bar carry** (`_commit_unwritten_history`) so `x[1]` is last bar's persist even when no `:=` ran this bar. On realtime ticks, `varip` re-evaluates the RHS.
`const` (v6)	Always initializes when the statement runs; not a cross-bar carry lock like `var` .
bare assign	Re-evaluates every bar; series "history" is the host series or prior list values.

Nested `var` inside `if barstate.islast` or a function therefore initializes on first path-taken bar—matching Pine's execution-based persistence.

`na` propagation

Operation	Rule
Binary arithmetic / comparison	Any `None` operand → `None` (or element-wise for lists)
Division by zero	→ `None` (not exception)
Soft type errors (`"a" + 1`)	→ `None`
Unary	`None` stays `None`
History OOB	→ `None`
Bare name `na`	Builtin / sentinel resolving to `None`
`nz(x, r)` (Numba path)	`nan` → replacement

List-valued series apply operators element-wise, aligning on the **trailing edge** when lengths differ (pad leading `None`).

`ta.highestbars` / `ta.lowestbars` offsets

These return a **bars-back offset** (not the extreme price):

Result	Meaning
`0`	Extreme is on the current bar
`-1`, `-2`, ...	Extreme is 1, 2, ... bars ago (down to `-(length-1)`)
`-1` sentinel (warm-up / invalid / all-`na`)	Short history (`bar_index+1 < length`), bad length, or no finite samples in the window

On ties, the **oldest** extreme in the window wins (leftmost bar). Interpret and Numba kernels share this contract (`_highestbars` / `numba_highestbars`).

History subscript interaction: Pine history indices are non-negative (`series[n]` with (n \ge 0)). Negative indices soft-fail to `na` on interpret series wrappers so warm-up / auto-step-`-1` / `highestbars` misuse do not abort the bar loop. On the compile path, dynamic offsets are coerced float→int (NaN→0) and the computed array index is clamped to `[0, n)` so expressions like `high[-ta.highestbars(...)]` (future-looking form) soft-fail to `nan` near series end rather than raising.

Bar mode vs full-series mode

Technical helpers (`technical_submodules/core.py`) distinguish:

- **Full-series mode** (unit tests with explicit lists): `ta.sma` may return a full list of values.
- **Bar mode** (`_pine_bar_mode`): returns the **current scalar** so expressions like `ta.ema(close,12) - ta.ema(close,26)` stay numeric per bar.

History buffers for indicators are truncated to a rolling window (`_SERIES_MAX = 256` for wrapper histories) to avoid (O(n^2)) full-history recomputation every bar.

Host series caps (`PYNE_SERIES_CAP`)

Pro API / `pynescript.runtime.Runtime` optionally **trims** chronological `current_series` lists and related host history so long charts do not grow (O(\text{bars})) memory per series.

Knob	Default	Meaning
`PYNE_SERIES_CAP`	**on**	Enable trimming. Disable with `0` / `false` / `no` / `off` (oracle / debug only).
Cap size	`256` (`DEFAULT_SERIES_MAX`)	Raised when the script declares a larger `max_bars_back=…`.
`PYNE_SERIES_MAX`	unset	Absolute override of the cap (positive int).
PineSeries history floor	`1000`	Separate from list caps; raised by `max_bars_back` / `PYNE_SERIES_MAX`.

Correctness notes (see `src/pynescript/runtime/series.py`; `backend/series.py` is a re-export shim):

- Window kernels (`ta.sma`, `highest`, ...) need `length ≤ cap`.
- Recursive smoothers under **incremental** TA (default `PYNE_TA_INCREMENTAL` on) carry state and stay safe independent of list length once warm.
- With **full recompute** (`PYNE_TA_INCREMENTAL=0`) and `bars ≫ cap`, EMA/RMA-style paths can diverge from a full-history oracle—prefer incremental (default) or raise the cap / disable series cap for goldens.

Out-of-range history offsets still return `na` (`None`); never `0`.

Unused derived series (0.3.10)

The interpret host updates `open` / `high` / `low` / `close` / `volume` / `time` every bar. Derived series (`hl2`, `hlc3`, `ohlc4`, `tr`, `time_close`) are written only when the source names them, uses `input.source`, or calls **`ta.vwap`** / **`vwap`** (default source is `hlc3` even when that identifier is absent). `ta.ao` rebuilds `hl2` from high/low if the derived list is empty.

`PYNE_SERIES_RING` (chronological tail view) remains **default off**. When on, the host skips the second chronological list write.

Internals

Path	Role
`src/pynescript/ast/evaluator/names.py`	`visit_Name`, `visit_Attribute`, `visit_Subscript`
`src/pynescript/ast/evaluator/expressions.py`	NA-safe binary/unary ops, series coerce
`src/pynescript/ast/evaluator/statements.py`	`var` / `varip` / `const` assign
`src/pynescript/ast/evaluator/builtins/technical_submodules/core.py`	`_as_series`, bar mode finalize
`src/pynescript/ast/evaluator/builtins/utility.py`	`max_bars_back`, `last_bar_index`, `na` helpers
`src/pynescript/runtime/host.py`	Bar loop; derived-series skip; series-cap trim (`backend.runtime` re-exports)
`src/pynescript/runtime/series.py`	`PineSeries`, `resolve_series_cap`, `PYNE_SERIES_CAP` / `PYNE_SERIES_MAX` / `PYNE_SERIES_RING`

Compile path: series are `np.full(n_bars, np.nan)` written at `__bar_idx`; history is `arr[__bar_idx - n]` with OOB → `np.nan` (see Compiler overview). Host **`time_arr`** is a chronological `float64` vector of bar-open Unix ms (same length as OHLCV); bare `time` lowers to `time_arr[__bar_idx]`, and `time[n]` uses the same history indexing rules as `close[n]`. When the host omits bar times, `Runtime` synthesizes `i * 60_000` so length always matches—calendar / `time` scripts stay defined on both modes.

Invariants & edge cases

1. **Chronology of `.history`.** Most-recent-first; reverse when converting to chronological lists for `ta.*` / `array.*`. Compile arrays are chronological (index 0 = first bar).
2. **No negative Pine indices.** Soft-fail to `na` / `nan` (not Python wraparound, and **not** a raise)—different from Python lists. See highestbars note above.
3. **`None` vs `float('nan')`.** Interpreter prefers `None`; Numba kernels use `np.nan`. Cross-mode comparisons must normalize.
4. **`max_bars_back`.** Raises the host series cap when larger than the default 256; still not a TV-style hard chart depth, but it affects trim policy.
5. **`PYNE_SERIES_CAP` default on.** Disable only for debugging / full-history oracles; pair with incremental TA for production long histories.
6. **Derived skip.** `ta.vwap()` still requires a live `hlc3` series; the host treats `vwap` as a derived-series consumer.

7. **Tuple returns from multi-value `ta.*`.** Unpacking `[a, b, c] = ta.macd(...)` uses `current` when it is a sequence of matching arity; otherwise history heuristics apply.
8. **Host series assign is by value.** New names never share the host OHLCV/`time` buffer reference.

Worked examples

History lag

```
//@version=6
indicator("lag")
delta = close - close[1]
plot(delta)
```

On bar 0, `close[1]` is `na` → `delta` is `na`. On bar 1+, arithmetic is ordinary floats (or series lists in batch tests).

`var` counter

```
//@version=6
indicator("count")
var int n = 0
n := n + 1
plot(n)
```

`n` initializes once; subsequent bars reassignment (`:=`) increments. Declaration sites are recorded in `_var_declarations`.

Element-wise NA

Given two list series of different lengths, `a + b` aligns tails and pads the longer prefix with `None`—preserving "most recent bars line up" intuition.

Failure modes

Symptom	Cause
All `na` after first bar	Host not updating series / `bar_index` between visits
`var` re-inits every bar	Fresh evaluator or `reset_var_declarations` each bar incorrectly
TypeError on `close + 1`	Series wrapper missing `.current` / not coerced
Compile vs interpret plot mismatch	`nan` vs `None`, or seed differences in EMA/RSI kernels — see parity
`time[j]` wrong after rebinding `last = time`	Expected only if host series were aliased; verify copy-on-assign path
`highestbars` always `-1`	Warm-up (`bar_index+1 < length`) or all-`na` window
Long-chart OOM / growing RSS	Series cap disabled or `PYNE_SERIES_MAX` huge; leave `PYNE_SERIES_CAP` on
EMA drift after many bars with cap	Full recompute + short cap — enable incremental TA or raise cap

See also

- Expressions & statements
- Technical builtins
- Numba path
- Interpret ↔ compile parity
- Runtime hub

EXPRESSIONS STATEMENTS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

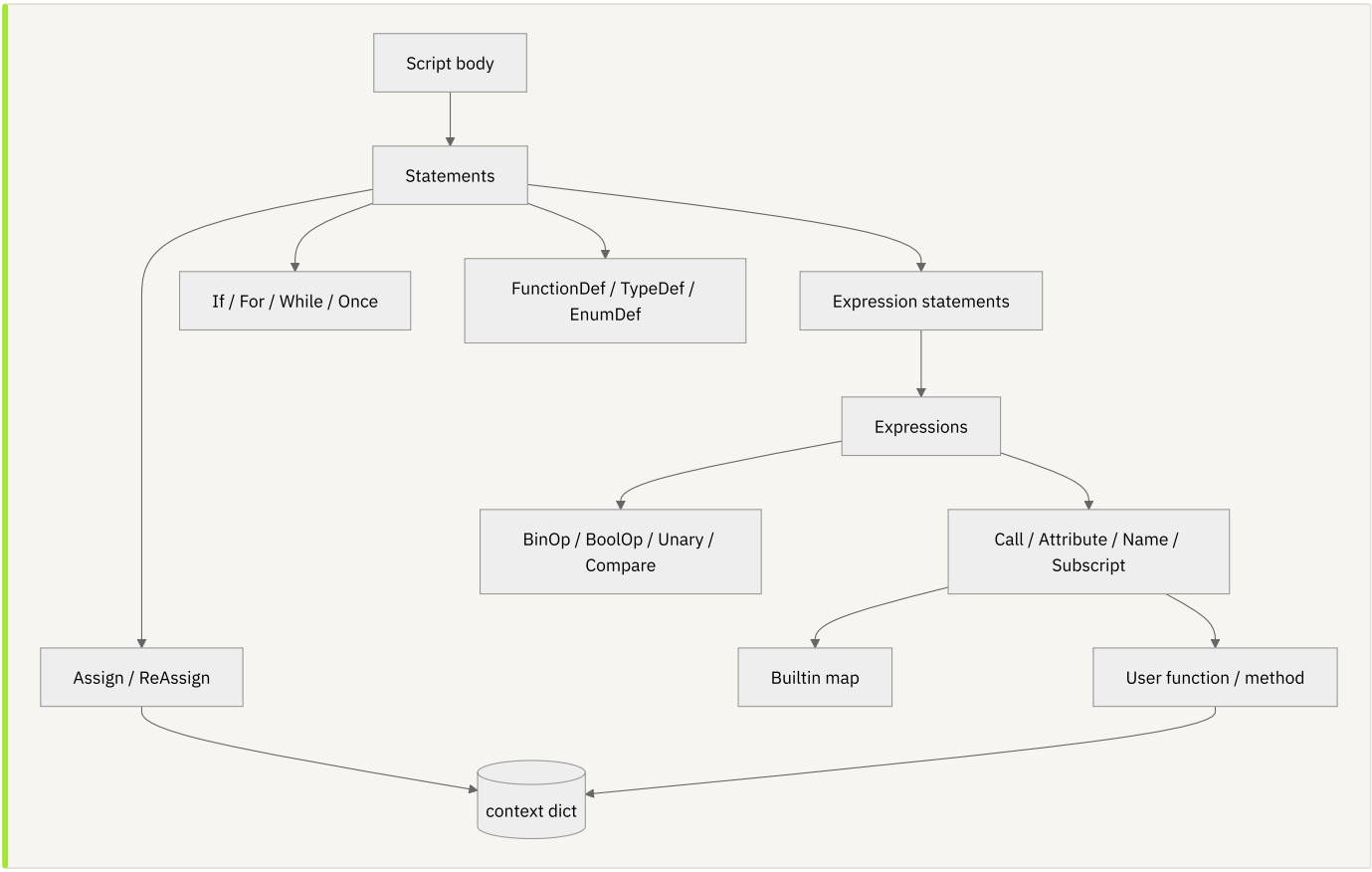
title: "Expressions and statements" description: "How the AST walker evaluates operations, calls, control flow, assignments, and user definitions."

EXPRESSIONS AND STATEMENTS

Abstract

Evaluation is pure visitor dispatch: each ASDL node type maps to a `visit_*` method on a mixin. Expressions produce values (with Pine `na` and series rules); statements mutate context, register types/functions, or emit side effects via builtins. This page is the operational semantics of the interpreter path—what runs inside `evaluator.visit(tree)` each bar.

Conceptual model



Interface surface

Expressions (`ExpressionEvaluator`)

Node	Behavior
`BoolOp` (`and` / `or`)	Manual short-circuit: `and` stops at first falsy (`na` / `0` / `False`); `or` stops at first truthy. Returns a Python `bool` .
`BinOp`	NA-safe `+ - * / %` ; division by zero → `na`
`UnaryOp`	`not` , unary `+` / `-` with NA and list map
`Compare`	Chained comparisons with NA-safe operators
`IfExp`	Ternary: condition, then, else
`Call`	Builtin dispatch, user functions, methods, multi-dispatch overloads
`Tuple` / lists	Literal sequences for unpacking and multi-arg returns

Binary ops coerce `PineSeries` -like objects to `.current` before operating. List operands broadcast or zip with trailing alignment (see Series & history).

Names (`NameEvaluator`)

Node	Behavior
`Name`	Context lookup; bare series builtins (`na` , `year` , ...); else string id (lazy)
`Attribute`	Qualified context keys, builtins (`strategy.position_size`), library modules, UDT fields/methods, array/drawing/matrix method markers, Python getattr fallback
`Subscript`	Series history / array / matrix indexing

Critical: qualified names like `strategy.opentrades.entry_price` are resolved via AST path construction (`ast_qualified_name`) **without** evaluating intermediate zero-arg series that would collapse the path to an int.

Statements (`StatementEvaluator`)

Construct	Behavior
`Script`	Walk body; finalize library exports if `library(...)` active
`Assign`	`var` / `varip` once-per-declaration; `const` ; ordinary assign; tuple unpack; `export` registration. History-tracked `var` series get a start-of-bar carry so `x[1]` is the prior persist.
`ReAssign` (`:=`)	Update existing binding (and UDT fields)
`If` / `For` / `While`	Standard control flow; loop variables in context
`Once`	Fire body the first time `test` is true (omit `test` → `true`) on a **confirmed** bar, then skip. Unconfirmed realtime ticks may re-run until `barstate.isconfirmed` commits the fired flag. Does not return a value.
`FunctionDef`	Store callable in context; multi-dispatch overload lists; `export` ; methods tagged `__pine_method__` . After bar 0 the host walks a **hot body** that skips Function/Type/Enum/Import (already bound).
`TypeDef` / `EnumDef`	Type registry / enum member dicts
`Import`	Resolve `LibraryRegistry` → bind alias to `LibraryModule`
Declarations	`indicator` / `strategy` / `library` via declaration builtins

Function invocation binds parameters into context (with restore of prior bindings on exit) so nested calls and bar-local state interact cleanly. After bar 0, hosts set `_pine_defs_locked` so re-visiting the script does not re-register overload tables. The locked walk inlines `Assign` and `Expr(Call)` (no generic `visit` frame) and UDF/call-expression history keys use `_pine_site_id` stamped on the AST.

Calls and multi-dispatch

Method markers returned from attribute evaluation:

Marker	Meaning
`("_method_call", instance, name)`	UDT method
`("_array_method", list, name)`	`array.<name>(self, …)`
`("_ns_method", obj, "label.get_text")`	Drawing/matrix namespace method
`("_ext_method", receiver, name)`	Free `method` with receiver as first arg

Overloads with typed first parameters (e.g. `matrix.float` vs `array.label`) select via coarse type tags; `na` receivers deliberately exclude matrix/array/drawing tags so optional UDT fields do not mis-dispatch to `tostring` on collections.

Internals

Path	Role
`src/pynescript/ast/evaluator/expressions.py`	Ops, calls, conditionals
`src/pynescript/ast/evaluator/names.py`	Names, attributes, subscripts
`src/pynescript/ast/evaluator/statements.py`	Script structure, assign, defs, import
`src/pynescript/ast/evaluator/literals.py`	Constants, colors, strings
`src/pynescript/ast/evaluator/base.py`	Context, math constants, library registry hook
`src/pynescript/ast/evaluator/types.py`	`EvaluatorProtocol` typing for mixins

Invariants & edge cases

1. **Context is the single store.** Builtins read OHLCV and strategy series from `self.context`; hosts must keep it coherent each bar.
2. **String fallback names.** An unbound `Name` returns its id string so later attribute/call resolution can still hit the builtin map (`"ta.sma"` patterns via attribute chains).
3. **Parameter unbind.** Missing-before-bind params use a sentinel so unbind `pop`s rather than leaving `None` ghosts.
4. **`na` normalization.** String `"na"` / `"nan"` / `"none"` may coerce to `None` at dispatch boundaries for UDT optional args.
5. **User functions re-enter the visitor.** Recursive and higher-order patterns are limited by Python stack, not a Pine-specific trampoline.

Worked examples

Ternary and comparison chain

```
//@version=6
indicator("cmp")
v = close > open ? close - open : open - close
plot(v)
```

Each operator path is NA-safe: if `close` or `open` were missing, comparisons and arithmetic yield `na`.

User function with series

```
//@version=6
indicator("fn")
f(x, n) =>
    ta.sma(x, n)
plot(f(close, 14))
```

`f` is stored as a Python callable that rebinds `x` / `n` and visits the function body AST each call.

UDF series parameters (0.6.2): when a caller passes a scalar (or a host-series current) into a function that then reads `s[1]`, interpret wraps that parameter as a per-call-site history stream. `src = close; updown(src)` therefore matches compile — the previous-bar sample is the previous argument at that call site, not an empty scalar. Official TradingView® builtin scripts are 0 MISMATCH vs compile on this contract.

`once` (August 2026)

```
//@version=6
indicator("once")
var int n = 0
once close > ta.sma(close, 20)
    n := 1
plot(n)
```

Equivalent to a `var bool` latch plus `if`, but cheaper: the structure is not re-evaluated after it fires on a closed bar. On a forming realtime bar (`Runtime.run(..., realtime_last_bar=True, realtime_ticks=N)`), the latch is not committed until `barstate.isconfirmed`. Compile is historical (`isconfirmed` always true).

Intrabar rollback on realtime ticks (0.6.0)

On a forming realtime bar, intermediate ticks snapshot the `var` scope (context bindings, series currents, declaration set) and restore it after the visit — every tick starts from confirmed state and only the final tick commits. `varip` names persist across ticks by design. Shallow by design: history pushes, the series-assign map, in-place container mutations, and broker side effects are not rolled back.

Method multi-dispatch sketch

```
method log(this, string s) => ...
method log(this, float x) => ...
```

Both register under the same name with `__pine_overloads__`; call sites pick by receiver/arg type tags.

Failure modes

Error	Meaning
`unexpected type of node`	Missing `visit_*` for an AST construct
`Unsupported binary operator`	Op class not in the dispatch table
History `series[-1]` is `na`	Negative Pine offsets **soft-fail** to `None` (not Python wraparound, not a raise)
Wrong overload / destroyed receiver	Historical `na` string path; now normalized—report residual mismatches as bugs
Stack overflow	Deep recursion in user functions

See also

- Series & history
- Builtins hub
- Libraries

- Type system (core)

OVERVIEW

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

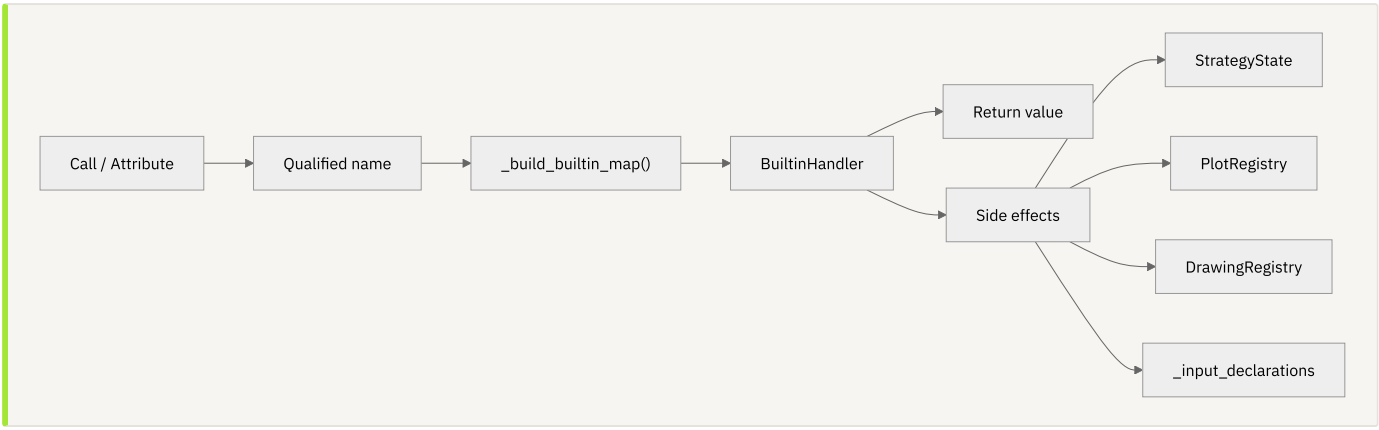
title: "Builtins" description: "Dispatch architecture for ta.*, strategy.*, collections, plotting, request.*, and other standard namespaces."

BUILTINS

Abstract

Pine's standard library is not a single module: it is a **flat qualified-name dispatch table** (`"ta.sma"`, `"strategy.entry"`, `"array.push"`, …) assembled from category mixins. Each entry is a handler `(args, kwargs?) → value` with side effects when the language requires them (orders, plots, drawings, inputs). This hub orients the surface; child pages go deep on technical, strategy, collections, drawing/plotting, and request/input.

Conceptual model



`BuiltinEvaluator` merges mixin maps in a fixed order and then registers ticker, logging, color, timeframe, and script-declaration functions. The `strategy()` declaration handler is wrapped so broker settings apply to `StrategyState`.

Interface surface

Namespace / area	Mixin / module	Docs
`ta.*`	`TechnicalAnalysisMixin` + `technical_submodules/*`	Technical
`strategy.*`	`StrategyBuiltinsMixin`, `StrategyConstantsMixin`	Strategy
`array.*` / `matrix.*` / `map.*`	`ArrayBuiltinsMixin`, matrix/map evaluators	Collections
`plot*` / `hline` / `line` / `label` / …	`PlottingFunctionsMixin`, `DrawingBuiltinsMixin`	Drawing & plotting
`request.*` / `input.*`	`RequestBuiltinsMixin`, `InputBuiltinsMixin`	Request & input
`math.*`, min/max, `nz`, …	`NumericBuiltinsMixin`	(this page)
`str.*`	`StringBuiltinsMixin`	(this page)
`color.*`	`register_color_functions`	(this page)
`syminfo` / `ticker.*`	`register_ticker_functions`	(this page)
`timeframe.*`	`register_timeframe_functions`	(this page)
`alert` / logging	`AlertsMixin`, `register_logging_functions`	Alerts
`indicator` / `strategy` / `library`	`register_script_declaration_functions`	Libraries
Footprint / volume_row	`FootprintBuiltinsMixin`	Request & input

Numeric and string essentials

- **Numeric:** `math.*` constants from `BaseEvaluator` (`math.pi`, `math.e`, golden-ratio helpers); functions cover abs, min/max, rounding, logs, trig, and NA helpers (`nz`, `na` predicates via utility).
- **String:** `str.*` formatting, conversion (`str.tostring` respects `format.*` constants), and manipulation aligned with common Pine scripts in the builtin corpus.

Color, ticker, timeframe

- **Colors:** hex constants (`color.red` → `#F23645`, etc.) plus composition helpers.

- **syminfo / ticker:** host-provided `Syminfo` object or flat context keys; ticker construction for `request.security` symbols.
- **timeframe:** period flags (`isintraday` , `isdaily` , ...) default daily in base constants; hosts override from bar spacing.

Alerts

`alert(message, freq)` and `alertcondition(condition, title, message)` record structured firings (`message` , `freq` , `bar_index` , `time` , `source`) with TV-style frequency rules (`once_per_bar` , `once_per_bar_close` , `all`). Hosts export them on `/run` and can POST last-bar batches to webhooks (**L2**). Side channel only — **not** strategy events.

Full detail: Alerts.

Internals

Path	Role
`src/pynescript/ast/evaluator/builtins/__init__.py`	`BuiltinEvaluator` composition
`src/pynescript/ast/evaluator/builtins/base.py`	Dispatch mixin, `_call_builtin` , registration
`src/pynescript/ast/evaluator/builtins/*.py`	Category implementations
`scripts/generate_builtin_metadata.py`	LSP/metadata surface (not hand-edited JSON)

Handlers are looked up by the fully qualified string built during name/attribute evaluation. Zero-arg series (e.g. `strategy.equity`) are registered as builtins that ignore empty args.

Invariants & edge cases

1. **One map, many mixins.** New builtins require a map entry *and* (usually) metadata regeneration for LSP.
2. **Kwargs and positionals.** Handlers accept both; Pine named arguments arrive as kwargs when the call AST carries them.
3. **Bar mode.** Stateful `ta.crossover` / similar use per-bar call indices reset by the host (`_cross_call_i`). Incremental `ta.*` (`PYNE_TA_INCREMENTAL` , default on) uses a sibling `_ta_call_i` slot.
4. **Mock fallbacks.** Bare equity-style `request.security` strings may still mock offline; foreign / fundamental prefixes and complex expressions return `na` (see request / input).
5. **Compile path subset.** Numba builtins reimplement only a fraction of this table; object mode covers more via Python helpers—see Compiler.

Worked example — resolution path

```
plot(ta.sma(close, 14))
```

1. Attribute/call chain builds qualified name `ta.sma` .
2. `_call_builtin("ta.sma", [close_series, 14])` .
3. Handler coerces series → list, computes SMA, finalizes scalar or list.
4. `plot` registers a `Plot` with that series value and returns the plot id.

Failure modes

Symptom	Fix
`Unknown built-in function`	Typo, version surface gap, or failed attribute chain
Always mock prices	Wire `data_feed` / `data_provider` into evaluator context
LSP knows a function runtime lacks	Metadata ahead of implementation—check missing features
Handler arity errors	Positional vs keyword mismatch in script

See also

- Technical
- Strategy
- Collections
- Drawing & plotting
- Request & input
- Alerts
- Builtin metadata (LSP)

TECHNICAL

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

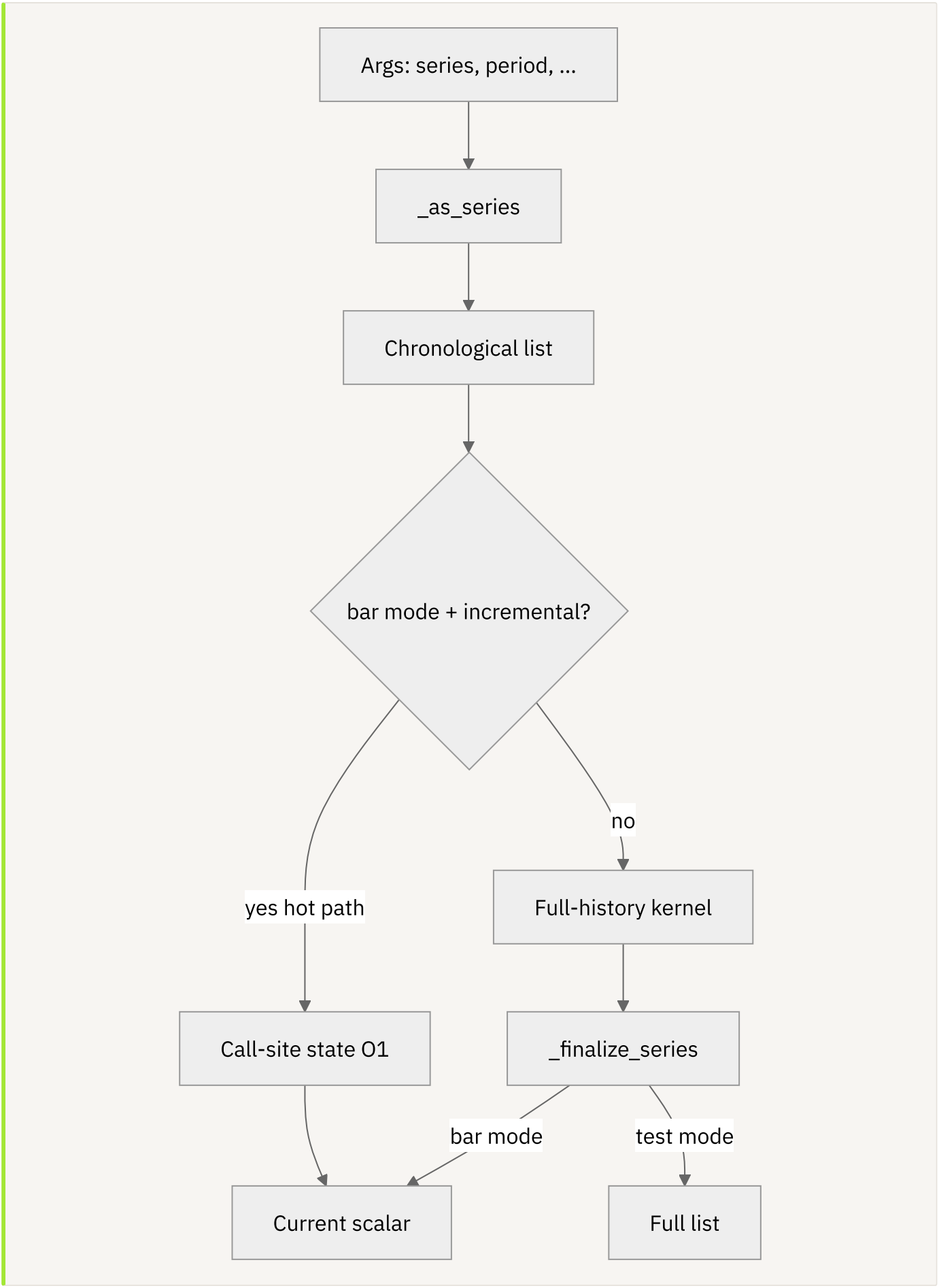
title: "Technical analysis (ta.*)" description: "ta.* indicators: series helpers, bar mode, interpret↔compile parity, and submodule layout."

TECHNICAL ANALYSIS (`ta.*`)

Abstract

The `ta.*` namespace is the largest pure-compute surface in PYNE: moving averages, oscillators, volatility bands, volume studies, pattern helpers, and cross/rise/fall predicates. Implementations live under `technical_submodules/` and are composed into `TechnicalAnalysisMixin`. Numerical behavior is validated against TradingView reference series (see numerical validation); the design goal is IEEE-limit parity, not “approximate TA.” Interpret and compile (Numba) hosts share the same formulas for the kernels listed under Interpret ↔ compile parity.

Conceptual model



Hosts in production set bar mode so indicator calls return **scalars for the active bar**, matching how Pine expressions compose (`ta.ema(a) - ta.ema(b)`).

Incremental hot path

When `_pine_bar_mode` and `_pine_ta_incremental` are enabled (Runtime default; disable with **`PYNE_TA_INCREMENTAL=0`**), hot `ta.*` builtins update **call-site state** once per bar instead of recomputing full history. Call sites are indexed like crossovers (`_ta_call_i` reset each bar). Nested forms such as `ta.ema(ta.sma(close, 14), 10)` stay correct because each call keeps its own slot. Golden suite: `tests/test_ta_incremental.py` (incremental last values ≡ full recompute).

Family	Incremental builtins (non-exhaustive)
Moving averages	`sma`, `ema`, `rma`, `wma`, `hma`, `vwma`, `swma`, `kama`, `dema`, `tema`, `alma`, `sma_weighted`
Oscillators	`rsi`, `macd`, `stoch` / `stochrsi`, `cci`, `cmo`, `tsi`, `roc`, `wpr`, `aroon`, `dpo`, `kst`
Volatility / structure	`atr`, `tr`, `stdev`, `bb`, `highest` / `lowest`, `linreg`, `adx` / `dmi`, `supertrend`, `kc`, `sar`, `donchian`
Volume (**0.3.10–0.3.11**)	`obv`, `wad` / `wvad`, `cmf`, `klinger`, `mfi`, `vwap`, `accdist`, `nvi`, `pvi`
Other	`change` / `mom` / `cum`, `median`, `percentrank`, `rising` / `falling`, `barssince`, pivots

`ta.nvi` / `ta.pvi` are incremental (O(1)/bar; `PYNE_TA_INCREMENTAL=0` full-list). **`ta.atr` is Wilder RMA of true range** on interpret (full + incremental) and Numba (`ta.rma(ta.tr, length)` — 0.3.4+). Supertrend is locked mid±factor·ATR (not the TV band ratchet).

Interface surface

Dispatch keys (non-exhaustive; map is authoritative in `technical.py`):

Family	Examples
Moving averages	`ta.sma`, `ta.ema`, `ta.wma`, `ta.rma`, `ta.hma`, `ta.vwma`, `ta.swma`, `ta.alma`, `ta.sma_weighted`
Oscillators	`ta.rsi`, `ta.stoch`, `ta.cci`, `ta.cmo`, `ta.mfi`, `ta.roc`, `ta.wpr`, `ta.tsi`, `ta.rci`
Trend / channels	`ta.macd`, `ta.adx`, `ta.dmi`, `ta.supertrend`, `ta.sar`, `ta.linreg`
Volatility	`ta.atr`, `ta.tr`, `ta.bb`, `ta.bbw`, `ta.kc`, `ta.kcw`, `ta.stdev`, `ta.variance`
Volume	`ta.obv`, `ta.vwap`, `ta.mfi` (shared), volume submodule helpers
Structure	`ta.highest` / `lowest` / `highestbars` / `lowestbars`, `ta.pivothigh` / `pivotlow`
Predicates	`ta.crossover`, `ta.crossunder`, `ta.cross`, `ta.rising`, `ta.falling`
Stats / other	`ta.change`, `ta.mom`, `ta.cum`, `ta.median`, `ta.mode`, percentiles, `ta.barssince`, `ta.valuewhen`, `ta.correlation`

Argument conventions

- Typical form: `ta.fn(series, length)`.
- Some functions allow **period-only** calls (e.g. `ta.highest(20)`) defaulting source to `high` / context series via `_expect_series(..., allow_period_only=True)`.

- Multi-value returns (e.g. MACD, BB, Supertrend) unpack as tuples/lists; assignment uses StatementEvaluator unpack rules.
- Fractional lengths floor to int (TradingView-compatible).

Stateful crosses

`ta.crossover` / `ta.crossunder` need previous-bar pairs. In bar-mode runs the host resets a call-site index (`_cross_call_i`) each bar so multiple cross calls in one script keep independent state.

Internals

Path	Role
`src/pynescript/ast/evaluator/builtins/technical.py`	Dispatch map aggregation
`.../technical_submodules/core.py`	Series coerce, expect helpers, bar finalize, **incremental kernels**
`.../moving_averages.py`, `oscillators.py`, `volatility.py`, `volume.py`, …	Kernels + inc wiring
`.../common.py`, `basic.py`, `advanced.py`, `patterns.py`, `strategies.py`, `synthesizer.py`, `economics.py`	Additional families
`src/pynescript/compiler/numba_builtins.py`	Compile-path Numba kernels (must track interpret formulas)
`tests/test_ta_indicators_*.py`, `tests/test_indicators.py`	Regression
`tests/test_ta_incremental.py`	Inc ≡ full recompute golden (full CI gate)
`tests/test_first_party_ta_goldens.py`	Dual-host first-party goldens (ATR / Supertrend / Keltner)
`tests/test_interp_compile_parity.py`	Always-on dual-host smoke + full-corpus mark
`scripts/compare_interp_compile.py`	Corpus harness (report under `.cache/interp_compile_parity.json`)
`tests/test_compiler_numba.py` (`TestInterpCompilePlotParityFixes`, highestbars offsets, …)	Targeted formula locks
`docs/numerical_validation_report.md`	Published precision summary (TV / IEEE bounds)

History for wrapper series is reversed to chronological order and may be truncated (`_SERIES_MAX`) before full kernels run. Incremental path only needs `series[-1]` per call, so truncation does not freeze state.

Interpret ↔ compile parity

Numba kernels are **separate code** from the interpreter, but the dual-host contract is **same formula, same warm-up mask, same `na` rules** for the surface below. Drift is a bug, not an allowed “approx compile.” Prove regressions with the interp↔compile harness and the numerical validation page (TV / IEEE bounds).

Topic	Shared behavior
`ta.rsi`	**Wilder** on both hosts: SMA seed of the first `period` deltas, then RMA of gains/losses. First finite bar is index `period` (`period` deltas need `period+1` prices). Not a simple window average.
`ta.roc`	Standard TV formula `100 * (src - src[length]) / src[length]` . Warm-up is **`na`** , never an early `0.0` . Zero or missing baseline → `na` . Interpret no longer used a wrong lookback denominator or early zero.
`ta.wma`	Requires a **full non-`na` window** of length `N` . Nested forms such as `ta.wma(ta.roc(...), …)` stay `na` until the inner series has produced `N` finite samples (no partial-window reweight).
`ta.cum`	Running sum; Pine **`na` / IEEE NaN treated as 0** (skipped contribution), matching TradingView cumulative-sum semantics.
`ta.highestbars` / **`ta.lowestbars`**	Return **negative bars-back offsets**: `0` if the extreme is the current bar, `-1` one bar ago, …, `-(length-1)` at the far edge. Short history (`i + 1 < length`) returns `-1` . Matches Aroon-style scripts that index with `high[ta.highestbars(...)]` / `math.abs(ta.lowestbars(...))` .
`math.avg` **(multi-arg)**	Arithmetic mean of the arguments. **Any `na` argument** → **`na`** (do not skip). Not a rolling `ta.sma` .
`ta.linreg`	Least-squares fit over the window; **endpoint at `x = n−1` when `offset=0`** (TV): `mean_y + slope * ((n−1) − mean_x)` / compile `intercept + slope * (n − 1 − offset)` . Length < 2 → `na` .
`ta.mfi`	Warm-up aligned on both hosts: needs **`length + 1` typical-price samples** (direction vs previous bar). Equal typical prices contribute to neither side; only-pos / only-neg MF → 100 / 0 (or 50 when both empty).
`ta.rci`	Rank Correlation Index (Spearman of time vs value ranks) is implemented on the **compile path** (`numba_rci`) as well as interpret; stays in numeric mode when called.

0.3.4+: `ta.atr` is Wilder RMA of TR on interpret + Numba (not EMA-of-TR). EMA dual-host seed is full-list SMA seed matching incremental/Numba. First-party dual-host goldens (ATR / Supertrend / Keltner) and `test_ta_incremental` gate residual drift. Any remaining gaps are tracked under missing features—not as silent “close enough” for the rows above.

Parity harness

```
# Always-on smoke (stable scripts under tests/test_interp_compile_parity.py)
pytest tests/test_interp_compile_parity.py -q

# Corpus compare (default ~50 scripts × 1000 bars; writes report JSON)
python scripts/compare_interp_compile.py --bars 1000 --limit 50
python scripts/compare_interp_compile.py --glob 'average_*.pine' --bars 200

# Optional longer pytest path
pytest tests/test_interp_compile_parity.py -m interp_compile_full

# Formula locks (RSI/ROC/WMA/cum/math.avg/highestbars, …)
pytest tests/test_compiler_numba.py -k 'InterpCompilePlotParity or highestbars_lowestbars_negative'
```

Report path: `.cache/interp_compile_parity.json` . Methodology and acceptable relative-error bands vs TradingView: Numerical validation.

Invariants & edge cases

1. **Warm-up → `na`.** Insufficient bars (e.g. SMA length (N) needs (N) samples; RSI needs `period+1` prices) yield `None` / leading `na` in full-series mode.
2. **NA in window.** Many kernels propagate `na` if any window element is missing (SMA-like sums, **WMA**, MFI money-flow samples). **`ta.cum` is the TV exception** (`na` → 0 contribution).
3. **Default sources.** `ta.atr` pulls high/low/close from `current_series` / context when not passed explicitly.
4. **Compile kernels must track interpret.** Numba `numba_*` implementations are separate source; for the parity table above they are formula-locked by dual-host tests—do not "approximate" for speed.
5. **No chart look-ahead.** Kernels only see history available at the current bar index; `request.security` lookahead is a separate concern.
6. **Incremental ≡ full recompute (oracle).** ATR seed is already Wilder RMA on both hosts; changing seed rules is a correctness project, not a silent perf flag.

Worked examples

Classic overlay

```
//@version=6
indicator("SMA 14")
v = ta.sma(close, 14)
plot(v)
```

In bar mode each visit returns one float (or `na`); the host stacks them into a series for the response envelope.

Cross entry signal

```
//@version=6
strategy("X")
fast = ta.ema(close, 12)
slow = ta.ema(close, 26)
if ta.crossover(fast, slow)
    strategy.entry("L", strategy.long)
```

Cross state is bar-local; ensure the runtime resets cross call indices when reusing an evaluator.

Multi-value unpack

```
[macdLine, signal, hist] = ta.macd(close, 12, 26, 9)
plot(hist)
```

Failure modes

Symptom	Cause
Always `na`	Length longer than available history; or series not updated
Wrong vs TV by large margin	Wrong source series; mock OHLCV; or non-bar-mode list composition bug
Cross never fires	Call-index state not reset / shared incorrectly across bars
Compile diverges from interpret	Kernel drift in `numba_builtins.py` (RSI must stay Wilder; ROC warm-up must stay `na`; WMA full window; highestbars sign; `math.avg` na rule)—re-run the parity harness
Early ROC / nested WMA finite too soon	Legacy quirks: ROC returned `0.0` before lookback, or WMA reweighted partial/`na` windows—fixed on both hosts
`high[ta.highestbars(...)]` mostly `na`	Offsets are **negative**; only offset `0` indexes the current bar without `math.abs`
`math.avg` finite when an arg is `na`	Bug: multi-arg avg must propagate `na`, not skip

See also

- Series & history
- Strategy builtins
- Numba path
- Numerical validation
- Missing features (residual dual-host gaps)
- Compiler overview

STRATEGY

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

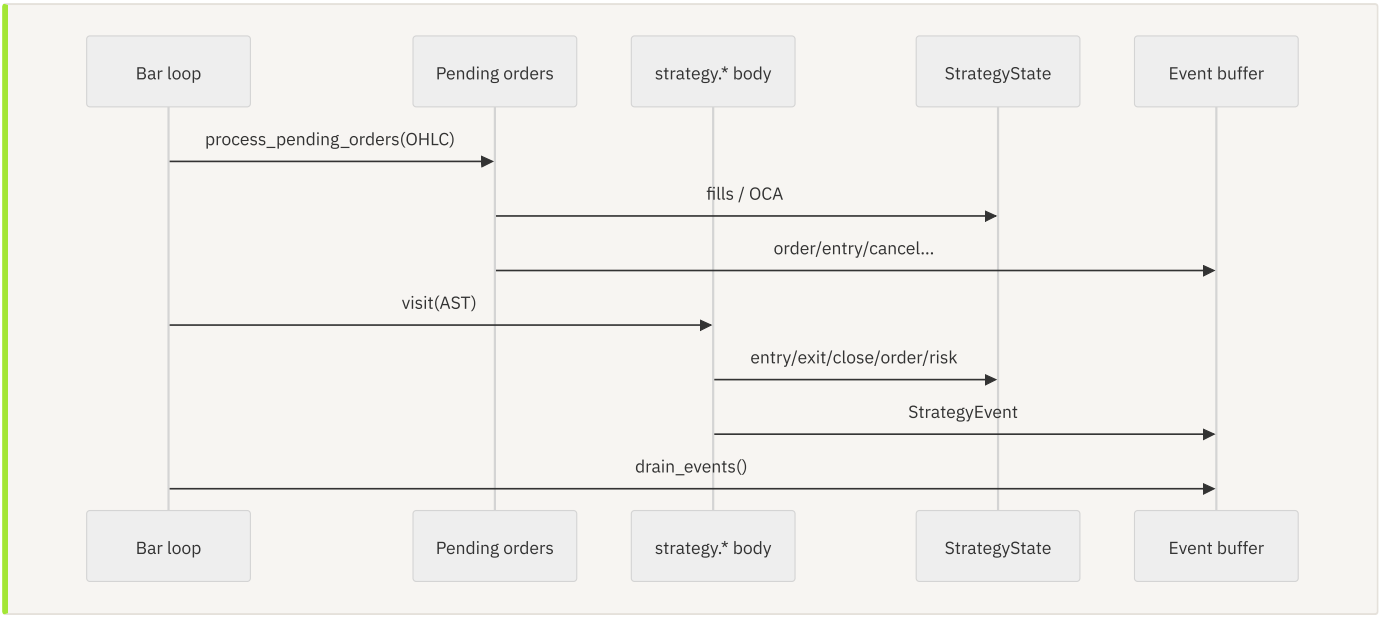
title: "Strategy builtins" description: "Orders, fills, OCA, commission, risk gates, position metrics, and StrategyState."

STRATEGY BUILTINS

Abstract

PYNE's strategy layer is a **per-run broker simulation** driven by `strategy.*` calls during the bar loop. It is not a live exchange adapter: orders become fills against bar OHLC (market immediately; limit/stop via pending book), commissions and slippage adjust PnL, and risk helpers can block entries. Structured StrategyEvent records form the parity contract with pyne-worker and the HOOX trade mesh.

Conceptual model



Fill-before-script matches the interpreter `Runtime` and the compile-path strategy broker.

Interface surface

Order placement

Builtin	Role
`strategy.entry`	Open/add (or reverse) by direction; market or pending limit/stop
`strategy.exit`	Bracket-style exit (pending limit/stop + OCA); market exit when no levels
`strategy.close` / `strategy.close_all`	Flatten by id or all (`qty_percent` supported)
`strategy.order`	Lower-level order with OCA name/type, partial fill cap
`strategy.cancel` / `strategy.cancel_all`	Remove pending orders

Directions accept Pine constants (`strategy.long` / `strategy.short`) and common string aliases.

`strategy.exit` surface (0.3.4+)

Feature	Status
Pending stop/limit brackets + OCA	Interpret + compile — fills via `process_pending_orders` OHLC path
`from_entry` multi-leg filter	Interpret + compile — unknown id is soft no-op
`qty_percent`	Interpret + compile `close` — `%` of target (whole pos or `from_entry`); wins over `qty` ; `≤0` /na → no-op; `>100` capped
Trail (`trail_offset` / `trail_points` / optional `trail_price`)	Interpret + compile — distances are **ticks** (`× mintick`). Pending stop ratchets from bar high/low (OHLC approx). `trail_points=0` / `na` is ignored so a valid `trail_offset` still applies (0.3.8).
`profit` / `loss`	**Ticks** from the exit's entry average (`ticks * mintick`). Long: `entry ± offset` ; short flips the sign. `na` / `≤0` ignore that leg. Absolute `limit` / `stop` win when both are set.

```
//@version=6
strategy("Exit demo", overlay=true, initial_capital=100000)
if bar_index == 5
    strategy.entry("L", strategy.long, qty=10)
if bar_index == 6
    strategy.exit("X", from_entry="L", stop=low * 0.98, limit=high * 1.02, qty_percent=50)
// Tick form (same surface): profit/loss are offsets from entry avg, not prices.
// strategy.exit("X", from_entry="L", profit=200, loss=100)
```

Position and performance series

Zero-arg builtins include `strategy.position_size` (signed: +long / −short), `position_avg_price`, `position_entry_name`, `opentrades` / `closedtrades` counts, `netprofit`, `openprofit`, `equity`, `cash`, gross win/loss stats, averages, max drawdown/runup, and max contracts held.

`strategy.position_avg_price` is the VWAP of **open** entry fills. A filled position is **not** overwritten when `strategy.entry` is called again with the same id (`if cond: strategy.entry("L", strategy.long)` stays at the first fill when `pyramiding=0`). Extra same-direction fills only occur when `opentrades < pyramiding + 1` (PYNE cap = `pyramiding + 1`; TV `pyramiding` default 1 ≡ PYNE 0). Pending limit/stop orders with that id are still replaced. When flat the series is `na`.

Trade queries

Indexed accessors:

- `strategy.closedtrades.entry_*` / `exit_*` / `profit` / `size` / `commission`
- `strategy.opentrades.entry_*` / `size` / `profit` / `commission`
- Open/closed trade fields include bar/time/id/comment plus approximate MAE/MFE (`max_drawdown` / `max_runup` from bar high/low)

Risk (interpret + compile halt cascade)

Builtin	Effect
`strategy.risk.max_position_size`	Cap size as % of equity (default); `PYNE_STRATEGY_RISK_CONTRACTS=1` for absolute contracts
`strategy.risk.max_intraday_loss`	Intraday loss gate — percent-of-initial-capital default, `strategy.cash` absolute (0.6.0, both brokers)
`strategy.risk.max_intraday_filled_orders`	Order count cap (day-scoped, enforced on both brokers)
`strategy.risk.max_drawdown`	Absolute / percent drawdown halt
`strategy.risk.max_cons_loss_days`	Consecutive losing calendar days → `entries_blocked`
`strategy.risk.allow_entry_in`	`"all"` | `"long"` | `"short"`

Blocked entries still emit a diagnostic-style event with comment `risk_blocked` where implemented. Compile broker shares the common halt cascade above (not a full TradingView® risk engine).

Declaration

`strategy(title, …)` applies broker settings onto `StrategyState`: `initial_capital`, commission type/value, slippage ticks, pyramiding, `avg_price_model`, etc.

Average price model (pynescript extension)

Not part of official TradingView Pine. Controls how `strategy.position_avg_price` evolves when size changes:

`avg_price_model`	On same-direction adds	On partial reduce
`"stock"` (default)	Arithmetic VWAP of fills	Re-average remaining open legs (FIFO multi-trade style)
`"futures"`	Same arithmetic VWAP	**Sticky** net AEP until flat (linear USDT-M / BTCUSDT-like)
`"inverse"`	Arithmetic today; harmonic add planned	Sticky until flat (same reduce rule as futures)

```
strategy("Perp style", avg_price_model="futures")
// or tokens:
// strategy(..., avg_price_model=strategy.avg_price_futures)
```

Linear add formula (stock and futures):

[avg' = \frac{avg \cdot |size| + p \cdot q}{|size| + q}]

Under `"futures"`, partial closes realize PnL against the sticky average (not per-leg FIFO entry prices), and closed-trade `entry_price` records that same average. Commission is never folded into the average numerator.

There is no `strategy.order.avg_future_price` series—use `strategy.position_avg_price` with the mode switch.

Leverage (pynescript extension — simpler futures UI)

Exchanges expose **leverage** (e.g. 5× / 10× / 20×). Prefer that over TV-style `margin_long` / `margin_short` percentages:

```
//@version=6
// TV-safe: strategy() kwargs need *const* (literals / const vars).
// Host UI can still inject runtime input overrides into PYNE.
strategy(
     "BTCUSDT style",
     avg_price_model="futures",
     leverage=10,
     default_qty_type=strategy.percent_of_equity,
     default_qty_value=100,  // use full equity as *margin*
     initial_capital=10_000)

// Optional Inputs-tab control for *trade logic* (not TV strategy Properties):
// lev = input.float(10, "Leverage", minval=1, maxval=125)
```

Setting	Effect
`leverage=N` (default `1`)	Buying power multiplier
`percent_of_equity` / `cash` default qty	`qty = margin × leverage / price`
`fixed` default qty	Unchanged (contracts as written)
`strategy.cash` / capital held	Margin locked = notional / leverage
`strategy.margin_liquidation_price`	Simple isolated estimate when `leverage > 1`
`strategy.leverage`	Read-back of configured multiplier

`leverage=10` also sets internal margin % to `100/10 = 10` (TV `margin_long` / `margin_short` equivalent). If only `margin_long` is set, leverage is derived as `100 / margin_long`.

`input` before `strategy()` — TV vs PYNE

	TradingView	PYNE
Statements before `strategy()`	Allowed for **const** vars (and types/enums/imports)	Allowed
`input.*()` return qualifier	**`input`** (not `const`)	Plain Python value (no qualifier system)
`strategy(leverage=input.float(...))`	**Compile error** — most `strategy()` params require **`const`**	**Interpret:** works. **Compile:** folds **const-like defval** into the broker ctor (`input.float(10)` → `leverage=10`)
User-adjustable leverage on TV	**Settings** → **Properties** ("Long/Short leverage", mapped from `margin_long` / `margin_short`)	Declaration `leverage=` or host-injected inputs

Qualifier hierarchy on TV: `const < input < simple < series`. A parameter that needs `const` **cannot** accept an `input` value. So even if `lev = input.float(10)` appears *before* `strategy()`, you still cannot pass `lev` into `strategy(...)` on TV.

PYNE does **not** enforce qualifiers; for compile-path parity with TV's const rule, non-const strategy kwargs are skipped unless their series folds to a constant defval.

Internals

`StrategyState` (`strategy.py`)

Per-evaluator instance fields include:

- Position: direction (`flat` / `long` / `short`), size (non-negative internal), entry metadata
- Books: `pending_orders`, `open_trades`, `closed_trades`
- Economics: capital, commission model, slippage, mintick
- Risk: max position %, drawdown, consecutive loss days, `entries_blocked`
- Equity curve peak/trough for max DD / runup
- `_events: list[StrategyEvent]` drained each bar

`signed_position_size()` implements Pine's signed `strategy.position_size`.

`Order` and fills

```
Order: market | limit | stop | stop-limit
  + oca_name / oca_type ∈ {none, cancel, reduce}
  + max_fill_per_bar (0 = fill remaining)
```

`process_pending_orders(open, high, low, close)` walks the book, computes trigger prices from OHLC (gap-aware open logic for limits/stops), applies commission/slippage, updates trades, runs OCA side effects, and emits events.

OCA

`oca_type`	On fill of a group member
`cancel`	Cancel other pending in group
`reduce`	Reduce sibling quantities by fill size
`none`	Independent

Covered by `tests/test_oca_commission.py` and order-fill suites.

Invariants & edge cases

1. **Isolation.** Never use class-level strategy state; concurrent runs must not share books.
2. **Pyramiding.** Additional entries respect `pyramiding` from the declaration.
3. **Partial fills.** `max_fill_per_bar` / defaults allow multi-bar completion of large orders.
4. **Calendar buckets for cons-loss.** Exit timestamps in ms vs seconds vs bar index are normalized in `note_closed_trade_day`.
5. **Compile path.** Object-mode uses `CompileStrategyBroker`—aligned fill rules, smaller surface than full interpreter metrics. Prefer interpret mode for full `strategy.closedtrades.*` analytics unless compile coverage is verified.

Worked examples

Market long / close

```
//@version=6
strategy("Demo", overlay=true, initial_capital=100000)
if bar_index == 5
    strategy.entry("L", strategy.long, qty=10)
if bar_index == 10
    strategy.close("L", qty_percent=100)
```

Parity fixtures under `tests/fixtures/parity/pine/` encode expected event sequences for such scripts.

Pending stop entry

```
//@version=6
strategy("Breakout", overlay=true)
strategy.entry("Break", strategy.long, stop=high[1])
```

Creates a pending stop; subsequent bars' `process_pending_orders` fill when `high` trades through the stop, using open-gap-aware fill prices.

OCA bracket sketch

```
//@version=6
strategy("Bracket", overlay=true)
// after entry "L" is open:
strategy.order("TP", strategy.short, qty=1, limit=tp, oca_name="br", oca_type=strategy.oca.cancel)
strategy.order("SL", strategy.short, qty=1, stop=sl, oca_name="br", oca_type=strategy.oca.cancel)
```

First fill cancels the sibling.

Failure modes

Symptom	Cause
Entry never fills	Pending stop/limit; OHLC never trades through; or risk block
Double entries	Pyramiding > 0 or missing close
Events missing `script_id`	Host forgot to stamp after `drain_events` (Runtime does this)
PnL off vs TV	Commission type, slippage ticks, mintick, or fill price model
Interpret vs compile event drift	Broker subset / timing—diff with `tests/test_compiler_strategy.py`

See also

- Events
- Strategy broker (compile)
- pyne-worker
- Pro API backtest

COLLECTIONS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

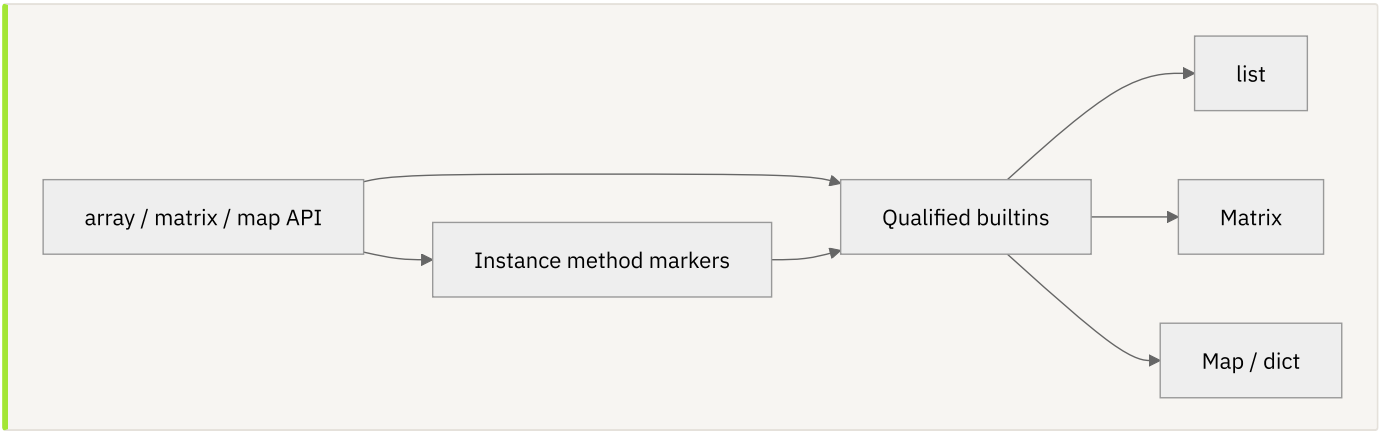
title: "Collections" description: "array.*, matrix.*, and map.* runtime semantics, method dispatch, and type tags."

COLLECTIONS

Abstract

Pine collections—**arrays**, **matrices**, and **maps**—are mutable values that live in the evaluator context (often under `var` so identity persists across bars). PYNE maps them to Python `list`, a dedicated `Matrix` type, and a `Map` / `dict`-like type respectively, with builtin functions and instance-method sugar (`a.push(x)` → `array.push(a, x)`).

Conceptual model



Interface surface

Arrays (`array.*`)

Represented as Python lists. Highlights from the dispatch map:

Category	Functions
Construct	`array.new`, `array.new_<type>`, `array.from`
Mutate	`push`, `pop`, `shift`, `unshift`, `insert`, `remove`, `set`, `fill`, `clear`, `reverse`, `sort`
Access	`get`, `first`, `last`, `size`, `includes`, `indexof`, `lastindexof`, `slice`
Aggregate	`sum`, `avg`, `min`, `max`, `median`, `mode`, `range`, `stdev`, `variance`, covariance, percentiles, percentrank
Search	`binary_search`, `binary_search_leftmost`, `binary_search_rightmost` — UDT arrays take `sort_field` (const int index, default 0, or const string name; array must be sorted by that field ascending)
Higher-order	`every`, `some`, `copy`, `concat`, `join`, `abs`

Indexing: Pine array indices are **0-based from the start of the list** (unlike series history). Negative indices are rejected for series; array paths use list semantics consistent with the builtin handlers.

UDT `sort_field` (0.3.6 / Pine August 2026): `array.sort`, `array.sort_indices`, and `array.binary_search*` compare one field of user-defined type elements. Pass a const int field index (`0` is the first declared field, and the default for search) or a const string field name. Binary search requires the array to already be sorted by that field in ascending order. Method form (`a.binary_search(value, "id")`) and `sort_field=` kwargs work on interpret and compile.

Series-like values passed into array ops unwrap via `.history` (reversed to chronological) when duck-typed.

Matrices (`matrix.*`)

`Matrix` supports:

- Construction: `matrix.new`
- Element access: `matrix.get` / `set`, and `m[row, col]` via subscript
- Shape: `rows`, `columns`, `elements_count`
- Row/column ops: add/remove/copy, sum/avg/min/max/mode, fill

- Aggregates: sum/avg/min/max/mode all
- Transforms: `transpose`, fill diagonal, and further linear helpers in the evaluator map

Method sugar: `m.rows()` → `matrix.rows(m)`.

Maps (`map.*`)

Function	Role
`map.new`	Empty map
`put` / `put_all` / `get` / `remove` / `clear`	Mutation
`contains` / `keys` / `values` / `size` / `copy`	Query

Keys follow Pine map rules as implemented (hashable Python keys). Compile object-mode lowers maps to ordinary Python `dict`s.

Internals

Path	Role
`src/pynescript/ast/evaluator/builtins/arrays.py`	Array builtins
`src/pynescript/ast/evaluator/builtins/matrix.py`	`Matrix` type
`src/pynescript/ast/evaluator/builtins/matrix_evaluator.py`	matrix.* handlers
`src/pynescript/ast/evaluator/builtins/map.py`	`Map` type
`src/pynescript/ast/evaluator/builtins/map_evaluator.py`	map.* handlers
`src/pynescript/ast/evaluator/names.py`	Method markers for list/Matrix
`tests/test_collections*.py`, `test_map_collections.py`, `test_matrix_*.py`	Coverage

Multi-dispatch type tags (`array.float`, `matrix.string`, …) interact with method overloading and `na` exclusion lists so `na`-typed optionals do not bind to collection `tostring` overloads.

Invariants & edge cases

1. **Identity under `var`.** `var a = array.new_float(0)` keeps one list across bars; reassigning `a := array.new_float(0)` replaces it.
2. **Drawing-typed arrays.** `array.new_label` etc. store drawing object references; delete semantics remain drawing-registry concerns.
3. **Empty array overload match.** Empty arrays match any `array.*` element tag in dispatch (no sample element).
4. **Matrix unpack hazard.** StatementEvaluator refuses to treat matrices as general iterables for tuple unpack—prevents corrupting multi-assign from row iteration.
5. **Compile mode.** Maps/UDTs force **object mode**; pure array numeric work may still numeric-compile depending on visitor detection—verify generated code when performance-critical.

Worked examples

Rolling buffer

```
//@version=6
indicator("buf")
var floats = array.new_float(0)
array.push(floats, close)
if array.size(floats) > 20
    array.shift(floats)
plot(array.avg(floats))
```

Matrix element

```
var m = matrix.new<float>(2, 2, 0.0)
matrix.set(m, 0, 1, close)
plot(matrix.get(m, 0, 1))
```

Map counter

```
var counts = map.new<string, int>()
map.put(counts, "n", nz(map.get(counts, "n")) + 1)
```

Binary search on a UDT array

```
//@version=6
indicator("udt search")
type Item
    int id
    float v
var a = array.new<Item>()
if bar_index == 0
    array.push(a, Item.new(2, 20.0))
    array.push(a, Item.new(1, 10.0))
    array.push(a, Item.new(3, 30.0))
    array.sort(a, order.ascending, "id")
plot(array.binary_search(a, 2, "id"))
plot(a.binary_search_leftmost(2, sort_field="id"))
```

Failure modes

Symptom	Cause
`map.get requires map and key`	Wrong arity or non-Map receiver
Index errors	OOB get/set without Pine `na` guard in that handler
Method not found	Typo or non-list receiver after series unwrap failed
Shared mutation across runs	Reused evaluator context without reset

See also

- Expressions & statements
- Drawing & plotting — arrays of labels/lines
- Compiler object mode

DRAWING PLOTTING

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

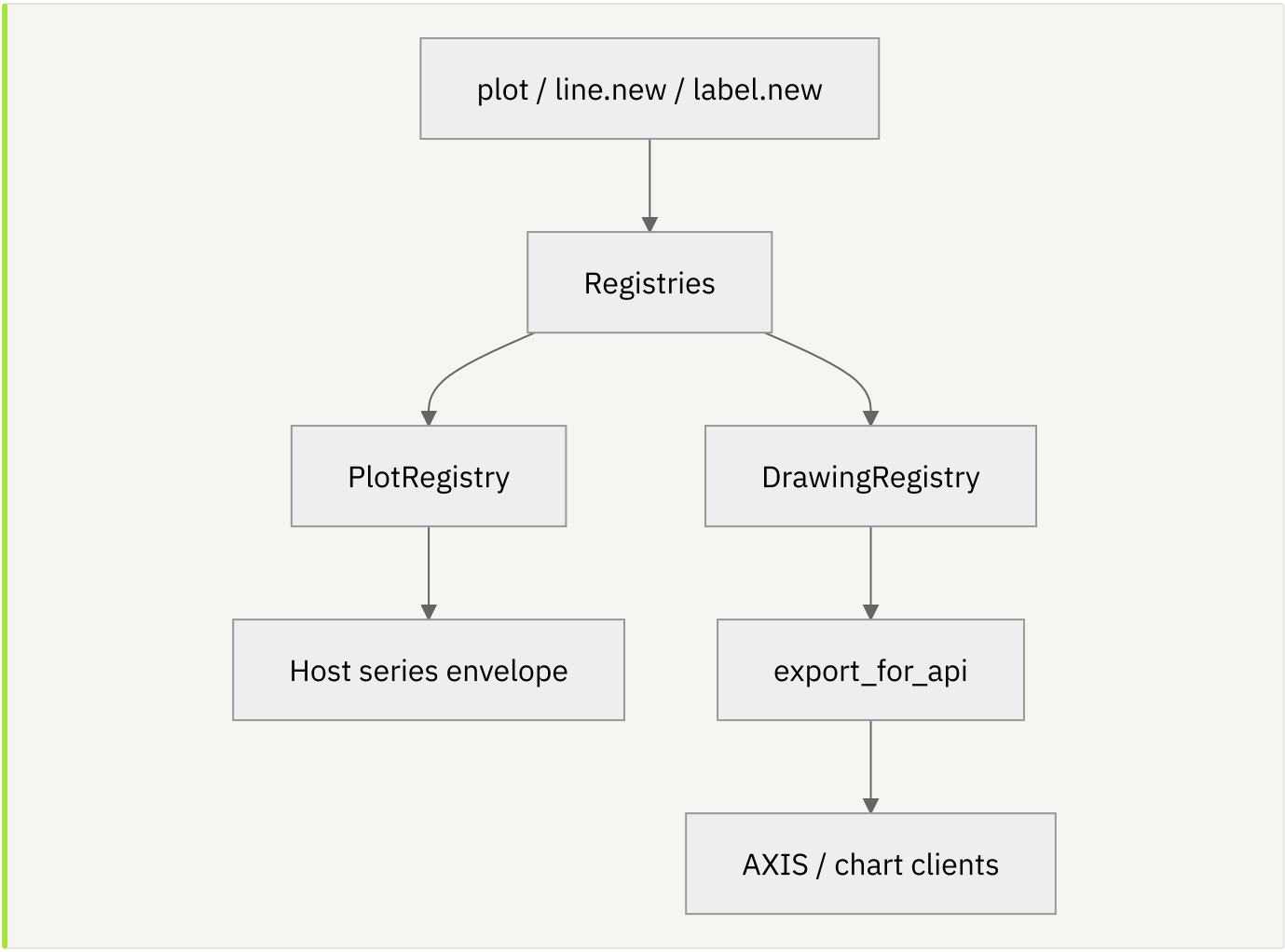
title: "Drawing and plotting" description: "plot* side effects, DrawingRegistry objects, and export shapes for hosts and AXIS."

DRAWING AND PLOTTING

Abstract

Visual effects in Pine are **side channels**: they do not change pure arithmetic results, but they are first-class runtime outputs. PYNE records plots in `PlotRegistry` and drawing primitives (lines, boxes, labels, tables, polylines, linefills) in `DrawingRegistry`, then serializes them for the Pro API and optional AXIS. Evaluation never requires a UI—tests assert on registries alone.

Conceptual model



Interface surface

Plotting (`PlottingFunctionsMixin`)

Function	Kind	Notes
`plot`	`plot`	Returns `Plot` id for `fill(plot1, plot2)` ; series values in host export
`plotshape` / `plotchar` / `plotarrow`	markers	Condition series + `plot_meta.kind` (style/location/char). Visuals are **not** in `drawings`
`barcolor` / `plotbar` / `plotcandle`	overlays	Per-bar color / OHLC series + kind stamp (0.3.12 dual-host)
`hline`	horizontal	**Both** hosts export a **constant price series** keyed by title + `plot_meta.kind: "hline"` . Not in `drawings`
`bgcolor` / `barcolor`	coloring	Color-string series + `plot_meta.kind` . Not in `drawings`
`fill`	fill	**Both** export a **titled series key** (values often all-null); band color + plot refs live in `plot_meta`

Style constants: `plot.linestyle_solid` / `dashed` / `dotted` . Plot style tokens: `plot.style_line` , `linebr` , `stepline` , `steplinebr` , `stepline_diamond` , `histogram` , `columns` , `cross` , `area` , `areabr` , `circles` .

Each call constructs a `Plot` dataclass (`kind`, `series`, `title`, `color`, linewidth, text formatting, force_overlay, ...) and appends it to `PlotRegistry.plots`.

Hosts often **also** capture per-bar plot values on the evaluator (`plot_outputs` / `_plot_value_cols`) for time-aligned series maps—see `src/pynescript/runtime/evaluator.py` and packaging in `src/pynescript/runtime/host.py` (`backend.evaluator` / `backend.runtime` re-export those modules).

`PYNE_LIGHT_PLOTS=1` skips columnar capture, plot-registry handles, and `input.*` metadata (corpus OK/fail only).

Title defaults and series keys

Runtime packages plots into `result["series"]` / `result["plot_meta"]` on **both** hosts. `result["drawings"]` is **geometry-only** (line / box / label / polyline / table / linefill). Compile `__drawings` is an internal event list that Runtime **folds and strips** before JSON — hosts never need it. Empty or missing titles get defaults; collisions get `_2`, `_3`, ... suffixes so map keys stay unique.

Call	Default title	Notes
`plot`	`plot_0`, `plot_1`, ...	Compile emits `plot_{n}` at emit time; interpret empty title → `plot_{index}` at JSON packaging
`hline`	`hline`	Uniquified `hline`, `hline_2`, ... when untitled levels repeat
`fill`	`fill`	Same uniquify rule (`Background` / `Background_2`, ...)
`bgcolor`	`bgcolor`	Series + `plot_meta` (both hosts)
`plotshape`	`shape`	Series + `plot_meta` (both hosts)

Explicit `title=` (or positional title) wins when present.

`hline` — constant series (both modes)

Interpret (`src/pynescript/runtime/evaluator.py` + Runtime packaging):

- Captures kind `hline` with the price as the series cell each bar.
- JSON export **forward-fills** nulls with the known constant so AXIS can draw a full-width level.
- `plot_meta[title]` includes `kind: "hline"`, `price`, `color`, `linewidth`, ...

Compile:

- Emits a **plot array** (`safe_float(price)` every bar) under the uniquified title and stamps `plot_meta.kind: "hline"`.
- May also append an internal `__drawings` event; Runtime packaging **strips** visual kinds so host `drawings` stays geometry-only.
- Host wraps NaN → null; constant levels remain numeric across the bar range.

Parity tests: `tests/test_multi_plot_cross.py` (`test_hline_series_keys_compile_matches_interpret`), `tests/test_plot_drawing_dual_host.py`, `tests/test_compiler_objects.py`.

`fill` — titled series key for key-set parity (both modes)

Interpret:

- Meta stores sibling series titles as `plot1` / `plot2` (resolved from `plot()` handles).
- Capture may hold a color string on the column; **Runtime JSON packaging** typically emits **all-null** series cells for fills (non-numeric / non-bgcolor strings → `null`). Band **color** and plot refs stay in `plot_meta` (and registry fill

objects when ids are enabled).

Compile:

- Registers a series key with expression `None` (float64 NaN → JSON `null`) under the uniquified title so **series key sets match interpret**.
- Band details (plot refs, color) land in `plot_meta`. An internal `__drawings` `fill` event, if emitted, is **stripped** before Runtime JSON.

Parity tests: `test_fill_background_series_keys_compile_matches_interpret`, `test_fill_title_series_key_and_drawings`, `tests/test_plot_drawing_dual_host.py`.

Import stubs must not leak as series strings

Unresolved `import …` libraries become chainable stubs marked `__pine_import_stub__ = True` (repr like `<PineImportStub namespace/name/version>`). If a script plots a stub member, Runtime's `_json_plot_value` maps those cells to **`null`**—never the stub repr or a color-like string—so AXIS and interpret/compile comparisons never see `"<PineImportStub …>"` as a plot series value.

Drawing objects (`DrawingBuiltinsMixin`)

Object types: `Line`, `Box`, `Label`, `Table`, `Polyline`, `LineFill`, `ChartPoint`.

Typical factories: `line.new`, `box.new`, `label.new`, `table.new`, `polyline.new`, `linefill.new`, plus getters/setters/deletes and `*.all` / last-bar helpers where implemented.

Instance methods resolve through namespace markers:

```
la.get_text()  →  label.get_text(la)
```

`DrawingRegistry.export_for_api(bar_times)` maps `xloc=bar_index` coordinates to wall times for chart clients, normalizes colors, and skips deleted objects. `xloc.bar_index` **past the last bar** (classic `bar_index + 1` on `barstate.islast`) is extrapolated from the series period (0.3.5); empty `bar_times` passes the bare bar index.

Line / box / label payloads include **`force_overlay`** for AXIS pane routing (0.3.6). `linefill.new` serializes as `type: "linefill"` quads (`t1` / `p1` … `t4` / `p4` from the two line endpoints, plus `color` / `bgcolor`).

Registry lifecycle

```
DrawingRegistry.reset()  # clears drawings + PlotRegistry
```

Hosts must reset at run start so labels from a previous evaluation do not leak.

Drawing GC (`max_*_count`)

Interpret path enforces TradingView®-style **garbage collection** on drawing factories (`line.new`, `label.new`, `box.new`, `polyline.new`, …):

Declaration kwargs (`indicator` / `strategy`)	Default	Hard cap
`max_lines_count`	50	500
`max_labels_count`	50	500
`max_boxes_count`	50	500
`max_polylines_count`	50	100

When more **active** objects of a type exist than the cap, the **oldest** are marked `deleted=True` so they leave `*.all` and `DrawingRegistry.export_for_api`. Caps are applied from the script declaration at run start (`DrawingRegistry.configure_from_declaration`); shrinking a cap immediately collects.

```
//@version=6
indicator("labels", overlay=true, max_labels_count=100)
if barstate.islast
    label.new(bar_index, high, "x")
```

Compile object-mode drawings are a simplified event list—GC caps apply on the **interpret** registry path; prefer interpret when you need full last-bar / `*.all` / cap behavior.

`fill` + `plot_meta` for AXIS

For dual `plot` + `fill` bands:

1. Each `plot()` contributes a **numeric series** under its title (default `plot_0`, ...).
2. `fill(p1, p2, …)` contributes a **titled series key** (often all-null cells) so key-sets stay stable, plus `plot_meta` that carries band color and plot refs.
3. Hosts such as AXIS should read `plot_meta[title]` for `kind`, colors, and sibling plot titles (`plot1` / `plot2`) rather than treating fill series as prices.

Interpret packaging lives in `src/pynescript/runtime/host.py` / `evaluator.py`; compile emits matching series keys. Internal `__drawings` fill events are folded/stripped before Runtime JSON.

Internals

Path	Role
`src/pynescript/ast/evaluator/builtins/plotting.py`	`Plot`, `PlotRegistry`, plot* handlers
`src/pynescript/ast/evaluator/builtins/drawing.py`	Drawing types, registry, builtins, `export_for_api`, `fold_compile_drawing_mutations`
`src/pynescript/runtime/evaluator.py`	Host plot capture: hline / fill / bgcolor / plotshape series + meta
`src/pynescript/runtime/host.py`	JSON packaging: series keys, stub nulling, hline fill-forward, fill plot refs
`src/pynescript/compiler/compiler.py`	`hline` / `fill` series + internal `__drawings`; visual events lifted then stripped
`src/pynescript/ast/evaluator/names.py`	`_DRAWING_METHOD_NS`
`tests/test_plotting_effects.py`, `test_drawing_all_and_last_bar.py`, `test_multi_plot_cross.py`, `test_bgcolor_plotshape_export.py`, `test_plot_drawing_dual_host.py`, `test_compiler_objects.py`	Behavior / parity

Compile object-mode notes

Geometry factories and some marker-style calls force **object mode** (pure-Python bar loop with an append-only `__drawings` event list). Runtime packaging **lifts** visual events into `series` + `plot_meta.kind`, then **strips** them so host `drawings` is geometry-only — the same AXIS shape as interpret `export_for_api`.

Call	Host export (both modes)	Compile internals
`hline`	Constant series + `plot_meta.kind`	Series array; optional `__drawings` event stripped
`fill`	All-null series key + `plot_meta` color/refs	Series key; optional `__drawings` event stripped
`plotshape` / `plotchar` / `plotarrow`	Condition/signed series + `plot_meta.kind`	Visual events lifted into series, then stripped
`bgcolor` / `barcolor`	Color-string series + `plot_meta.kind`	Same lift/strip as other visuals
`plotbar` / `plotcandle`	OHLC series + kind stamp	Same lift/strip
`label` / `line` / `box` / `polyline` / `table` / `linefill`	Geometry objects in `drawings`	Handle dict + `__drawings`; `set_*` / `*.delete` fold via `DrawingRegistry.fold_compile_drawing_mutations`

Numeric mode still supports plain `plot` float64 arrays; geometry or marker calls can flip object mode for that script.

Invariants & edge cases

1. **`plot` returns an id.** Required for `fill`; treating it as “void” breaks band fills. Hosts that skip `PlotRegistry` unless `fill(` appears in source must still soft-fail fill gracefully.

2. **Deleted flag.** Soft-delete keeps history for debugging; `active()` filters deleted plots and GC-collected drawings.
3. **`max_*_count` GC.** Oldest active drawings are deleted when caps are exceeded (interpret registry).
4. **Series in drawings.** Prices may be series wrappers—export coerces `.current` and maps NaN → omit.
5. **Compile path.** Numeric mode supports `plot` arrays; geometry/markers force object mode with an internal `__drawings` event list. Runtime **folds** `set_*` / `*.delete`, **lifts** visual events into `series` + `plot_meta.kind`, then **strips** them so `drawings` is geometry-only. Prefer interpret when you need full last-bar / `*.all` / registry GC — not because visuals lack series.
6. **Import stubs → null.** `__pine_import_stub__` and `"<PineImportStub…>"` strings never appear as plot series cells after Runtime packaging.
7. **No GPU/canvas in-process.** Registries are data; AXIS/HOOX render elsewhere.

Worked examples

Dual plot + fill

```
//@version=6
indicator("BB mid")
basis = ta.sma(close, 20)
p1 = plot(basis)
p2 = plot(basis * 1.01)
fill(p1, p2, color=color.new(color.blue, 80))
```

Label on last bar

```
//@version=6
indicator("lbl", overlay=true)
if barstate.islast
    label.new(bar_index, high, str.tostring(close))
```

Hosts read `DrawingRegistry.labels` or API export after the run.

Failure modes

Symptom	Cause
Empty drawings in API	Forgot `DrawingRegistry.reset` timing / export; or script never called `*.new`
fill no-ops	plot ids not retained from `plot()` returns
Stale labels across HTTP runs	Missing registry reset between `Runtime.run` calls (Runtime resets—custom hosts must)
Object mode missing drawing	Compile still folds `set_*` / `delete`; leftover events usually mean fold did not run
Only 50 labels/lines visible	Default `max_*_count`; raise on `indicator` / `strategy` (≤ hard cap)
fill series all null	Expected for band keys; use `plot_meta` for color + plot refs

See also

- Builtins hub

- Pro API preview / chart renderer
- AXIS docs
- Compiler overview

REQUEST INPUT

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "request.* and input.*" description: "Multi-symbol/timeframe requests, fundamental na semantics, host series bind, and input parameter resolution."

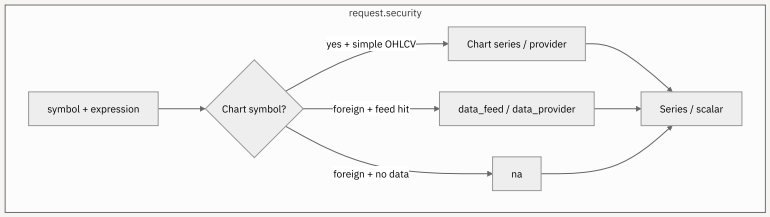
`request.*` AND `input.*`

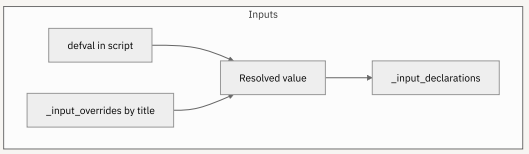
Abstract

Two namespaces couple scripts to the **host environment**: `input.*` declares parameters (with defaults and UI metadata), and `request.*` pulls data from other symbols, timeframes, or fundamental/economic sources. PYNE implements both as builtins with explicit extension points— `data_feed` , `data_provider` , and `_input_overrides` — so the same script can run offline or online with real market data.

Design rule: missing multi-symbol or fundamental data yields **`na`** , never silent substitution of chart OHLCV (no close-as-dividend, no chart volume as UPVOL).

Conceptual model





Interface surface

`input.*` (`InputBuiltinsMixin`)

Builtin	Purpose
`input`	Generic defval + title/tooltip/inline/group/confirm/active
`input.bool` / `int` / `float` / `string` / `color`	Typed scalars
`input.price` / `source` / `time` / `timeframe` / `session` / `symbol`	Domain inputs
`input.enum` / `input.text_area`	Enumerations and multiline text

Runtime value: each call returns the resolved value (override if title matches, else default). Metadata is appended to `_input_declarations` for settings panels and LSP-adjacent hosts.

Overrides live on the evaluator as `_input_overrides: dict[title, value]`.

`request.*` (`RequestBuiltinsMixin`)

Builtin	Role
`request.security`	Other symbol / timeframe expression
`request.security_lower_tf`	Lower-TF array expansion
`request.dividends` / `earnings` / `splits`	Corporate actions
`request.financial` / `economic` / `quandl`	Fundamentals / external series
`request.currency_rate`	FX conversion helper
`request.seed`	Deterministic pseudo-series
`request.footprint`	Volume footprint object (v6 surface)

`request.security` — interpret path

Resolution order (simplified):

1. Normalize symbol (series → last element; `ticker.*` → symbol string) and timeframe.
2. Classify **chart vs foreign** (`_is_chart_symbol` vs host `syminfo` / provider symbol).
3. Try `data_feed.fetch_latest_ohlcv` / ticker, then `data_provider.fetch`.
4. **Same-symbol coarser timeframe (HTF resample, 0.3.4+):** bucket chart OHLCV to the requested TF; **last completed** HTF bar by default (lookahead_off-style). Allowlisted simple TA on HTF series — `ta.sma` / `ta.ema` /

`ta.rsi` / `ta.atr` / `ta.wma` / `ta.rma` (bare `close` / `high` /... source + const length; `ta.atr(n)` length-only) — evaluates on the unique completed HTF series. Nested `ta.*`, UDFs, and non-const lengths stay `na`. `barmerge.gaps_on` / `lookahead_on` are honored on these resample paths (0.6.0, finalized-bucket lookahead in 0.6.1): gaps deliver the value only on bucket-start bars, lookahead leaks each HTF bucket's final value from the period start on historical bars. Both stay unused on passthrough / provider / complex-na paths. Policy meta is exposed on `result["meta"]["request_security"]` (`gaps_supported` / `lookahead_supported`, `gaps_applied` / `lookahead_applied`).

5. **Foreign + pre-evaluated expression** (UDF result, list/tuple of chart values, non-string expr) without multi-symbol data → **`na`** (not chart close as “dividends”).
6. Fundamental / non-equity prefixes (`DIVIDEND`, `FACTSET`, `EARNINGS`, `ESD_`) with no feed hit → **`na`** (no mock OHLCV).
7. Bare equity-style string names may still use **legacy mock** prices for offline demos when no feed is wired.

`ChartOHLCVProvider` (wired by `Runtime` via `resolve_request_sources` when unset) **only serves the chart ticker**. Foreign tickers get empty series, so interpret returns `na` unless a real multi-symbol feed is configured.

`request.security` — compile path

Compile lowering is intentionally narrow (compiler overview):

Case	Emit
Same-symbol (`syminfo.ticker` / empty chart id) **and** simple OHLCV expr (`close`, `high[1]`, ...)	Passthrough chart array sample
Foreign tickers (`UPVOL.NY`, `ESD_FACTSET`, ...)	`np.nan`
Complex third arg (UDFs, `year_sum(close)`, non-allowlisted `ta.*`, arithmetic)	`np.nan`
Other `request.*` APIs	`np.nan` (object mode)

No inventing chart-close-as-dividends or chart-volume-as-advance/decline series.

Footprint (`FootprintBuiltinsMixin`)

Types `Footprint` and `VolumeRow` expose buy/sell volume, delta, VAH/VAL/POC rows, and per-row imbalance helpers—aligned with the v6 footprint surface inventory.

Mode selection (`mode=auto`)

Pro API `/run` defaults body `mode` to **`auto`**. `Runtime._compile_eligible` **rejects compile** when the source contains `request.` (or top-level `import`), so auto **prefers interpret** for any script that uses `request.*`:

```
request.* present → compile_fallback_reason = "request.* not supported in compile path"
                  → auto_backend = interpret
```

Forced `mode=compile` still runs the narrow lowering above (simple same-symbol only; else `na`). Input overrides also force interpret under auto.

See Compiler overview for warm compile, numeric vs object mode, and eligibility.

Host series bind (interpret)

Assigning host OHLCV / time series into a **user name** never aliases the host buffer by reference. `_bind_series_name` copies the current scalar into a fresh series for the user name so patterns like:

```
last_t := na(close) ? last_t[1] : time
```

cannot corrupt `time[j]` history. That bug previously broke TTM / `year_sum`-style windows (e.g. dividend-yield scripts).

Internals

Path	Role
`src/pynescript/ast/evaluator/builtins/input.py`	Input handlers + declarations
`src/pynescript/ast/evaluator/builtins/request.py`	`request.*` + chart/foreign / na policy
`src/pynescript/ast/evaluator/statements.py`	`_bind_series_name` (no OHLCV alias)
`src/pynescript/util/data.py`	`ChartOHLCVProvider`, `resolve_request_sources`
`src/pynescript/compiler/compiler.py`	Same-symbol OHLCV-only `security` lower
`src/pynescript/runtime/host.py`	Package Runtime SoT; feed wiring; `_compile_eligible` / `_run_auto`; copies policy onto `meta.request_security`
`backend/runtime.py`	Compat re-export of package Runtime (not the implementation)
`tests/test_dividend_yield_parity.py`, `test_datafeed_wiring.py`	na parity + chart provider

Invariants & edge cases

1. **Inputs are pure values at runtime.** Titles matter only for override keys and UI metadata—not for Pine type identity.
2. **Foreign without data → `na`.** Prefer honest missing data over inventing chart series as multi-asset results.
3. **Chart provider is chart-only.** Multi-asset accuracy needs a real `data_feed` / `data_provider`.
4. **Mocks are limited.** Equity-style bare symbols may still mock offline; fundamental prefixes and foreign pre-evaluated exprs do not.
5. **Dynamic symbols.** List/series symbols resolve to the latest element—supports loops constructing ticker ids.
6. **Lower TF.** `request.security_lower_tf` returns array-like structures; length scales with simulated lower-TF density when mocking.
7. **`mode=auto` + `request.*` → interpret.** Compile path is not a full multi-asset substitute (`_compile_eligible` rejects `request.`).
8. **HTF is last-completed only.** Forming HTF buckets are never returned; `gaps` / `lookahead` kwargs do not change that.

Worked examples

Parameterized length

```
//@version=6
indicator("len")
len = input.int(14, "Length", minval=1)
plot(ta.sma(close, len))
```

Host:

```
ev._input_overrides = {"Length": 21}
```

Multi-timeframe close / simple HTF TA (chart symbol)

```
//@version=6
indicator("HTF")
htf = request.security(syminfo.tickerid, "D", close)
htf_sma = request.security(syminfo.tickerid, "D", ta.sma(close, 20))
plot(htf)
plot(htf_sma)
```

Same-symbol coarser TF: interpret buckets chart OHLCV (last **completed** HTF bar) and evaluates allowlisted simple `ta.*` (`sma` / `ema` / `rsi` / `atr`) on that series. Compile may passthrough simple same-symbol OHLCV; complex foreign/security still → `na` . Inspect `meta.request_security.policies` (`htf_ohlcv_resample` , `htf_simple_ta_resample` , `foreign_na` , `complex_htf_na` , ...).

Intentional `na` — dividend yield (fundamentals missing)

```
//@version=6
indicator("div")
year_sum(src) =>
    ta.cum(src)
div_ticker = ticker.new("ESD_FACTSET", "X;Y;DIVIDENDS")
div_ttm = request.security(div_ticker, "D", year_sum(close), barmerge.gaps_on, lookahead=barmerge.l
plot(div_ttm)
```

Without a fundamentals feed: **interpret and compile both plot `na`** for `div_ttm` —not chart close as fake TTM dividends. Covered by `tests/test_dividend_yield_parity.py` .

Intentional `na` — CVI / UPVOL-style foreign OHLCV

```
//@version=6
indicator("cvi")
up = request.security("UPVOL.NY", "D", close)
plot(up)
```

Foreign ticker + no multi-symbol data → `na` . Compile never rewrites this as chart close; inventing advance/decline volume from the host series is forbidden.

Failure modes

Symptom	Cause
All-`na` multi-asset plots	Foreign symbol / fundamental expr; no feed (expected)
Always ~100 mock prices	Bare equity string + no feed; legacy mock path
Input override ignored	Title string mismatch (including empty title)
Footprint fields zero	`request.footprint` without configured footprint data
`time[1]` / history wrong after assign	Should be fixed: host bind no longer aliases OHLCV
Lookahead surprises	Host data alignment / gaps—not automatic TV replay guarantees
`auto_backend=interpret` with `request.*`	Eligibility prefilter; use interpret or wire multi-symbol feed

See also

- Compiler overview — auto eligibility, same-symbol security lower
- Builtins hub
- Series & history
- Pro API run endpoint
- Runtime hub

LIBRARIES

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

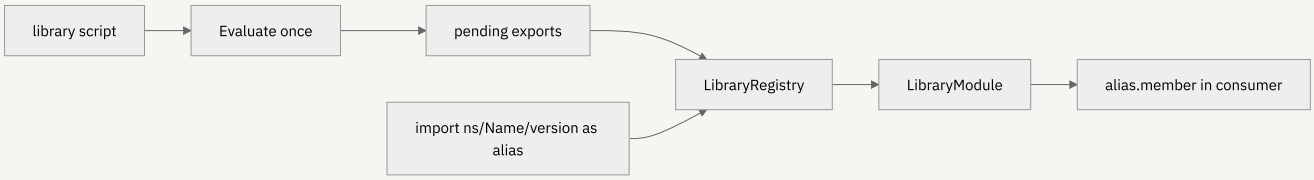
title: "Libraries" description: "In-process library export/import, LibraryRegistry, and evaluate-order requirements."

LIBRARIES

Abstract

TradingView libraries publish as `namespace/Name/version` with `export`ed members. PYNE implements an **in-process** analogue: evaluate a `library("Title")` script (or register source by path), collect exports (`export const`, exported functions, types), then `import` binds an alias to a `LibraryModule` for `alias.member` access. There is no network publish step—the registry is the host's responsibility.

Conceptual model



Interface surface

Library script

```
//@version=6
library("MyMath")

    a + b
```

On script completion, `StatementEvaluator` finalizes `_pending_library_exports` onto the active `LibraryModule` and calls `LibraryRegistry.register`.

Consumer import

```
//@version=6
indicator("use")

plot(M.add(close, M.PHI))
```

Resolution (`LibraryRegistry.lookup`):

1. Exact `(namespace, name, version)` if provided and registered.
2. Else title-only match (`MyMath`) for local evaluation workflows without publisher path.

API on the evaluator

```
ev.register_library_source(namespace, name, version, source_str)
mod = ev.lookup_library(namespace=..., name=..., version=...)
```

`register_library_source` stores Pine text for **lazy** load on the first matching `import`. `_load_library_source` evaluates definitions only (skips showcase body) and **finalizes** `_pending_library_exports` onto the `LibraryModule` so `export const` / exported functions bind (0.3.8). Evaluating a library script on the same evaluator still populates exports eagerly.

Host evaluate (`0.3.7+`)

Hosts pass published sources without a pre-eval step:

```
Runtime(symbol="TEST").run(
    consumer_source,
    ohlcv,
    mode="interpret",  # or "auto" — import is compile-ineligible
    libraries=[
        {"namespace": "ns", "name": "Lib", "version": 1, "source": library_source},
    ],
)
```

The same list is accepted on Pro API and **pyne-worker** `POST /run` as `libraries`. pyne-worker also stores the array on deployed scripts so cron / `script_id` reuse it. Pro API slices to **32** entries (`backend/app.py`); `Runtime.run` itself does not cap. `mode=compile` has no import path — `mode=auto` treats `import` as compile-ineligible and **forwards `libraries=` into interpret fallback** (0.3.9).

`LibraryModule`

Dataclass with `title`, optional `namespace` / `version`, and `exports: dict[str, Any]`. Attribute access reads exports; missing members raise `AttributeError` with a clear message. `NameEvaluator` routes `alias.member` when the base value is a `LibraryModule`.

Internals

Path	Role
`src/pynescript/ast/evaluator/libraries.py`	`LibraryModule`, `LibraryRegistry`
`src/pynescript/ast/evaluator/statements.py`	export registration, import visit, library finalize
`src/pynescript/ast/evaluator/base.py`	registry fields on `BaseEvaluator`
`src/pynescript/ast/evaluator/__init__.py`	`register_library_source`, `lookup_library`
`src/pynescript/runtime/host.py`	`Runtime.run(..., libraries=)` + auto-mode forward
`tests/test_library_export_import.py`	Round-trip coverage

Exportable values include callables (user functions), constants, and type-related bindings depending on declaration paths. Exported methods participate in the same call machinery as local functions once retrieved from the module.

Invariants & edge cases

1. **Evaluate library before consumer** (or register source for lazy import). Empty registry → import failure.
2. **Same evaluator instance** (or shared registry) must see both scripts—registries are not process-global unless the host shares them.
3. **Version is part of the path key.** Mismatched version does not fall back to another version automatically when namespace is specified.
4. **Title registration** always updates `_by_title` so simple local titles work without publisher metadata.
5. **No TV account auth.** Publishing/versioning outside the process is out of scope.

Worked examples

Explicit registration

```
from pynescript.ast.evaluator import NodeLiteralEvaluator

ev = NodeLiteralEvaluator()
ev.evaluate_script(library_source)   # registers by title
ev.evaluate_script(consumer_source)  # import by title or path
```

Path registration without pre-eval

```
ev.register_library_source("user", "MyMath", 1, library_source)
# import user/MyMath/1 loads definitions and finalizes exports on first visit
```

Failure modes

Symptom	Cause
Import resolves to nothing	Library never registered / wrong title
`Library 'X' has no exported member 'y'`	Missing `export` or typo
Stale exports	Re-evaluated library without re-import; host cache
Version conflict	Multiple libs same title—last `register` wins for title key

See also

- Expressions & statements
- Builtins declarations
- End-user library API guide

EVENTS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

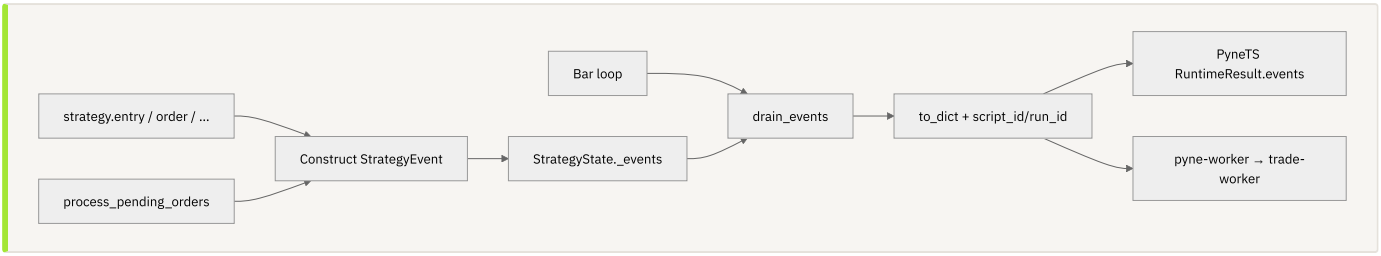
title: "Strategy events" description: "StrategyEvent shape, emission points, parity corpus, and host serialization."

STRATEGY EVENTS

Abstract

A **strategy event** is the immutable, structured record of one `strategy.*` action (or fill side-effect) at a specific bar. Events are the wire format between PYNE's Python evaluator, **PyneTS** (`StrategyEvent` on `RuntimeResult`), and downstream HOOX trade workers (via **pyne-worker**). Changing the dataclass without updating the TS library and tests is a contract break.

Conceptual model



Interface surface

`StrategyEvent` fields

Defined in `src/pynescript/ast/evaluator/events.py`:

Field	Type (logical)	Notes
`kind`	`entry` | `exit` | `close` | `close_all` | `cancel` | `cancel_all` | `order`	Discriminator
`id`	`str | None`	Order / entry id
`direction`	`long` | `short` | `None`	
`qty`	`float | None`	
`order_type`	`market` | `limit` | `stop` | `None`	Stop-limit may appear as composite handling elsewhere
`limit` / `stop`	`float | None`	Prices when relevant
`oca_name`	`str | None`	OCA group
`comment`	`str | None`	Includes system comments (`risk_blocked` , `fill:…`)
`bar_index`	`int`	From context at emit
`bar_time`	`int`	From context
`ohlc`	4-tuple → list in JSON	Bar snapshot
`script_id`	`str`	Host-stamped (source hash)
`run_id`	`str`	Host-stamped run uuid

Frozen dataclass → `to_dict()` always includes keys (JSON nulls for unspecified fields) so parity diffs are stable.

Emission sources

1. **Explicit builtins** — `strategy.entry` , `exit` , `close` , `close_all` , `cancel` , `cancel_all` , `order` push events as they execute.
2. **Broker fills** — `process_pending_orders` emits fill-related `order` / `entry` / `cancel` (OCA) events.
3. **Risk gates** — blocked entries may emit with `comment="risk_blocked"` .

Host collection pattern

```
evaluator.reset_events()
# ... process_pending_orders + visit(tree)
for ev in evaluator._strategy_state.drain_events():
    d = ev.to_dict()
    d["script_id"] = script_id
    d["run_id"] = run_id
    all_events.append(d)
```

`Runtime.run` implements this and returns events in the response envelope.

Compile path

`CompileStrategyBroker` accumulates plain dict events (`__events` in the return mapping) with the same field names for kinds it supports. Exit/trail/OCA/risk halt comments (`risk_blocked`) are on both hosts for the 0.3.4+ surface; remaining metric accessors still trail the interpreter.

Internals

Path	Role
`src/pynescript/ast/evaluator/events.py`	`StrategyEvent`
`src/pynescript/ast/evaluator/builtins/strategy.py`	Emit sites + `drain_events`
`src/pynescript/compiler/strategy_broker.py`	Compile-mode `_emit`
`tests/test_strategy_events.py`	Unit behavior
`tests/test_parity.py` + `tests/fixtures/parity/`	Cross-port oracle
`pynets/src/runtime/interpret.ts` `StrategyEvent`	TS library twin

Parity corpus scripts (`strategy_01_entry_long.pine` , ...) regenerate expected JSON via:

```
python tests/fixtures/parity/generate_fixtures.py
```

Tests strip `script_id` / `run_id` before compare.

Invariants & edge cases

1. **Immutability.** Do not mutate events after construction; drain copies and clears the buffer.
2. **Per-bar reset.** `reset_events` / drain prevents cross-bar leakage in tests that share evaluators.
3. **OHLC list vs tuple.** `to_dict` converts to list for JSON round-trips.
4. **Kind vocabulary is closed.** New kinds require TS + fixtures + this doc.
5. **Not alerts.** `alert()` / `alertcondition()` use a separate alerts channel—not `StrategyEvent` .

Worked examples

Expected single entry

Script enters long on a fixed bar; fixture JSON lists one event:

```
{
  "kind": "entry",
  "id": "L",
  "direction": "long",
  "qty": 1.0,
  "order_type": "market",
  "bar_index": 5
}
```

(plus nullables and ohlc—see real fixtures for full keys).

Cancel after OCA

Fill of one OCA sibling yields `cancel` events for others with `comment` reflecting `oca_cancel` / `oca_reduce`.

Failure modes

Symptom	Cause
Empty events with live strategy	Forgot drain; or strategy never called
Parity flake on ids	Comparing `script_id` / `run_id`
TS/Python mismatch	Field rename without dual update
Duplicate events	Not clearing buffer; double `process_pending_orders`

See also

- Strategy builtins
- Strategy broker
- PyneTS
- pyne-worker
- HOOX live trading
- HOOX docs

ALERTS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Alerts" description: "alert() / alertcondition() engine, host export, and L2 HTTP webhooks (Pro API + pyne-worker)."

ALERTS

Abstract

Pine `alert()` and `alertcondition()` fire into a **host side-channel**, not strategy events. The evaluator records structured firings with TradingView-style frequency rules. Hosts (Pro API and **pyne-worker**) export them on evaluate responses and can **POST** them to HTTP webhooks (roadmap **L2**).

Conceptual model



Layer	Module
Engine	`src/pynescript/ast/evaluator/builtins/alerts.py` (`export_alerts` / `export_alerts_from_evaluator`)
Package host	`src/pynescript/runtime/host.py` (`clear_alerts` per run, `alerts` on the envelope)
Pro API webhook	`backend/alert_forwarder.py` (`backend/runtime.py` is only a Runtime re-export)
Edge host	sibling pyne-worker `alert_engine.py` + `alert_forwarder.py`

Pine surface

`alert(message, freq)`

- Default `freq` : `alert.freq_once_per_bar` (canonical token `once_per_bar`).
- Constants (also bare strings): `alert.freq_once_per_bar` , `alert.freq_once_per_bar_close` , `alert.freq_all` .
- **once_per_bar** — first fire per bar for the same (source, title, message, freq) key.
- **once_per_bar_close** — only when the host marks the bar confirmed (`barstate.isconfirmed` / `islast` ; hosts without barstate treat bars as confirmed).
- **all** — every call.

`alertcondition(condition, title, message)`

- Records each evaluation in `alert_conditions` (debug / UI).
- When `condition` is true, also **fires** a host alert with `source: "alertcondition"` and once-per-bar semantics.

Event shape (`to_dict` / API)

```
{
  "message": "cross up",
  "freq": "once_per_bar",
  "bar_index": 42,
  "time": 1700000000000,
  "title": "optional",
  "source": "alert"
}
```

`source` is `"alert"` or `"alertcondition"` . Hosts stamp `script_id` / `run_id` (and edge cron may add `symbol` / `timeframe`).

Host export

Pro API (`POST /run`)

- Interpret path: full history of firings in `alerts` (and optional `alert_conditions`).
- Compile path: `alerts: []` today (alert side effects are interpret-only).
- See POST /run.

pyne-worker

- `POST /run` returns the same `alerts` array.
- Cron filters to the **last closed bar** before webhook delivery so historical bars do not re-fire every minute.
- Health: `features.alerts`, `features.alert_webhooks`.

L2 webhooks

Both hosts POST JSON to an HTTPS endpoint.

Host	Default URL	Per-request / per-job
Pro API	env `ALERT_WEBHOOK_URL`	body `webhook_url`
pyne-worker	secret/var `ALERT_WEBHOOK_URL`	script/job `webhook_url`

Pro API request flags

Field	Default	Meaning
`webhook_url`	`""`	Override destination
`forward_alerts`	`true`	Skip POST when false
`alert_last_bar`	`true`	Only firings on the last OHLCV bar
`alert_batch`	`true`	One batch body vs one POST per alert

Optional timeout: env `ALERT_WEBHOOK_TIMEOUT` (seconds, default 10).

Batch body

```
{
  "type": "pine_alert_batch",
  "source": "pyne-pro-api",
  "count": 1,
  "content": "cross up",
  "alerts": [
    {
      "type": "pine_alert",
      "source": "pyne-pro-api",
      "message": "cross up",
      "freq": "once_per_bar",
      "alert_source": "alert",
      "bar_index": 42,
      "time": 1700000000000
    }
  ]
}
```

Edge worker uses `"source": "pyne-worker"`. Discord-friendly `content` is set when a message is present.

Response meta

Successful `/run` may include:

```
"alert_forward": {
  "forwarded": 1,
  "failed": 0,
  "filter": "last_bar",
  "url": "https://hooks.example.com/pine",
  "batch": true,
  "count": 1
}
```

Webhook failures never fail the evaluate status; inspect `alert_forward` / `alert_forward_error`.

Worked example

```
curl -sS -X POST http://127.0.0.1:5002/run \
  -H 'Content-Type: application/json' \
  -d '{
    "script": "//@version=6\nindicator(\"a\")\nif bar_index == last_bar_index\n    alert(\"fire\")\
    "mode": "interpret",
    "webhook_url": "https://hooks.example.com/pine",
    "data": [
      {"time": 1, "open": 1, "high": 2, "low": 0.5, "close": 1.5, "volume": 10},
      {"time": 2, "open": 1.5, "high": 2.5, "low": 1.0, "close": 2.0, "volume": 12}
    ]
  }'
```

Invariants

1. **Not strategy events** — trade mesh uses `events[]`; alerts are independent.

2. **Per-run clear** — hosts call `clear_alerts()` so runs do not leak firings.
3. **Last-bar default for webhooks** — full-history still available in the `alerts` response field.
4. **Compile path** — prefer `mode: "interpret"` (or auto fallback) when you need alerts.

See also

- Strategy events
- POST /run
- Pro API usage
- Runtime bridge
- Roadmap (L2 ✅)

OVERVIEW

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

---

title: "Compiler overview" description: "Source-to-source compilation: CompilerVisitor, numeric vs object mode, run(time=...), and engine API."

COMPILER OVERVIEW

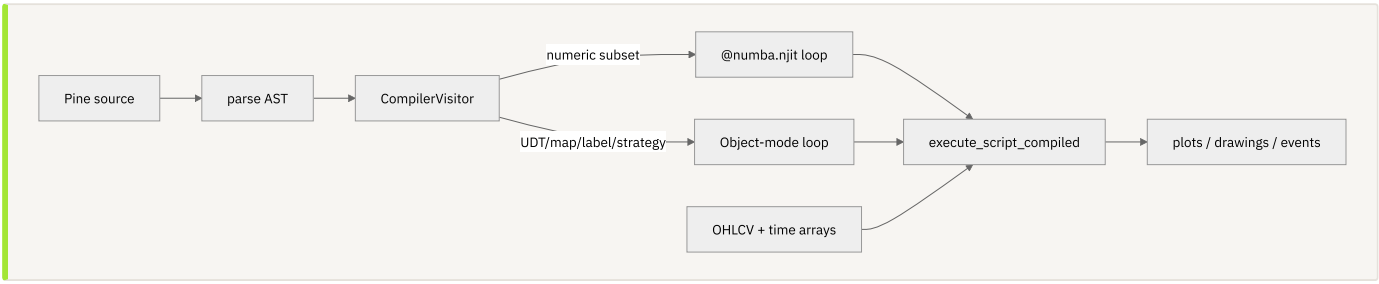
Abstract

The compile path lowers Pine's ASDL AST to **executable Python** that walks bars with an explicit index (`__bar_idx`) over contiguous `numpy` arrays—avoiding per-node visitor overhead. Pure numeric scripts wrap the loop in `@numba.njit`; scripts that touch UDTs, maps, drawings, or strategy select **object mode** (a tight Python/numpy loop, no AST walk). **Numba is required only for numeric mode**; object mode runs with numpy alone. **`once`** (0.4.4) lowers to a persistent `__once_fired_N` flag; UDF-scoped `once` forces object mode.

The public façade is `pynescript.compiler.engine` (`transpile`, `compile_script`, `run_script`), also reachable as `Runtime.run(..., mode="compile")` or `mode="auto"`.

This is an MVP with growing surface—not a full substitute for the interpreter on every script (notably multi-asset `request.*` and full strategy analytics). **0.3.16–0.3.17** recovered a large object-mode residual class: UDF locals, nopython `None`/unicode helpers, matrix handles, UDT returns, statement `switch` assign, drawing/`chart.point` copy, `round()` extras, import stubs as `na`, color type dispatch, and nested UDT method fields. Official TV builtins are **0 MISMATCH** vs interpret (**P1p** closed on that set); optional set0x community corpus is unmeasured, not a missing language surface.

Conceptual model



Generated entry point is always:

```
execute_script_compiled(open_arr, high_arr, low_arr, close_arr, vol_arr, time_arr)
```

Host packing lives in `engine.CompiledScript.run` (and `pynescript.runtime.Runtime._run_compiled`).

Interface surface

```
from pynescript.compiler.engine import transpile, compile_script, run_script, has_numba

src = transpile(pine_text)           # generated Python string
cs = compile_script(pine_text)       # CompiledScript
result = cs.run(open, high, low, close, volume, time=None)
# result: dict of plot title → float64 array, optional __drawings, __events, …
```

API	Role
`transpile`	Parse + visit → source string (inspect/debug)
`compile_script`	`exec` generated code, warm-up call, return `CompiledScript`
`run_script`	One-shot compile+run (prefer cache `CompiledScript` for batches)
`has_numba`	Whether numeric (njit) mode can load
`prewarm_numba_builtins` / `prewarm_scripts`	Host cold-start (H2 warm path)
`compile_cache_stats` / `compile_deploy_config`	Cache + deploy diagnostics
`clear_compile_cache`	In-process LRU only
`clear_disk_compile_cache`	Disk IR / index under cache dir
`clear_numba_function_caches`	Purge Numba `.nbi` / `.nbc` under known dirs

`CompiledScript` fields: `source`, `generated_code`, `execute`, `plot_titles`, `plot_kinds` (per-plot `plot` / `hline` / `fill` tags used to synthesize numeric-mode `__drawings`), `object_mode`, optional `nopython_fallback_reason`.

`CompiledScript.run` signature

```
def run(
    self,
    open_, high, low, close,
    volume=None,   # default: ones
    time=None,     # bar-open Unix ms; optional 6th series
) -> dict[str, Any]:
```

Argument	Behavior
OHLCV	Coerced to contiguous `float64`; lengths must match (`ValueError` otherwise)
`volume=None`	Filled with ones
`time=None`	Synthetic `bar_index * 60_000` ms (unit-test / pure-compile default)
`time=arr`	Real bar-open timestamps; length must match OHLCV

`Runtime._run_compiled` packs OHLCV dicts and **passes real bar times** so calendar/`time` / `timestamp` / `time[n]` stay aligned with interpret. Pure `cs.run(o,h,l,c)` without `time=` is fine for pure-price scripts and fixtures.

When `time` is omitted, the engine synthesizes `np.arange(n) * 60000.0` (1-minute bar opens from epoch). Hosts that care about `year` / `month` / `time[n]` must pass bar-open Unix ms.

Warm compile (product defaults)

Pro API `/run` defaults to **`mode=auto`** (compile when eligible, interpret on fallback with `compile_fallback_reason`). Disk IR cache is **on** (`PYNE_COMPILE_DISK_CACHE=1`); Docker sets `PYNE_COMPILE_CACHE_DIR=/data/compile-cache`. Operators call `POST /compile/prewarm` or `pynescript prewarm` so first interactive latency skips Numba cold JIT. Design notes live in repo `docs/COMPILER_PLAN.md` (not a product page).

`Runtime._compile_eligible` (used by `mode=auto`) **skips compile** when the source contains a top-level `import` or any `request.` token — those stay interpret even though the emitter can lower same-symbol `request.security`. Explicit `mode=compile` still attempts the visitor.

Mode selection

`CompilerVisitor` sets `object_mode = True` when it sees:

- User-defined types / enums
- Map / array / matrix handles that cannot stay float64
- Drawing constructors (`label.new`, `line.new`, …) and assigned `hline` handles
- Strategy usage (object mode + `CompileStrategyBroker`)
- String/color series, library imports, or other non-numeric constructs

Statement-form `hline(price)` / `fill(...)` with a proven-numeric price **stay nopython**: they emit a constant (or all-`nan`) series plus `plot_kinds` metadata; the engine synthesizes `__drawings` after the JIT loop. Mixed scripts that later flip object mode replay those events into the body.

Otherwise **numeric mode** is preferred. Numeric mode **requires Numba** at load/warm-up; missing Numba raises `CompileNumbaRequiredError`. Object mode does **not** need Numba (still star-imports `numba_builtins` for `safe_*` / non-jitted helpers).

On numeric warm-up **TypingError** / nopython failure, the engine re-emits with `force_object_mode=True` and records `nopython_fallback_reason`—so `mode="compile"` can still run without hard-failing on partial numeric coverage.

Numba is **not** a hard requirement for all compile. Object-mode scripts (strategy, drawings, UDT, maps) execute without it. Only a pure-numeric emit that never flips to object mode needs Numba installed.

What numeric mode lowers today

Series assigns, arithmetic/logic, history (`close[n]`), `if` / `for` / `while`, selected `ta.*` (and bare TA aliases), scalar math helpers, `plot`, `input.*` defaults, `var` / `varip` carry—executed under `@numba.njit(cache=False)` on the generated entry (`cache=False` because code is `exec`'d from a string without a stable disk locator). Disk-cached IR modules may rewrite to `cache=True` for cross-process reuse of machine code.

Object mode extras

- UDT instances as field dicts
- Maps as Python dicts
- `__drawings` list of structured events
- Optional `__strategy` broker: pending fills, position, `__events`, equity snapshots

Compile-path limits (notable)

Construct	Compile behavior
`request.security` / bare `security` / `request.security_lower_tf` / `request.seed`	**Same-symbol, simple OHLCV expression only** (chart identity + bare `close` / `high` /... style). Foreign symbol or complex expression → `np.nan` (no inventing chart close as foreign data). Other `request.*` → object-mode `np.nan` stub
Bare builtins vs user series	User-defined series arrays win in `visit_Name`: `ad = ta.cum(...)` / `tr = …` shadow bare `ad` / `tr` formulas so `plot(ad)` is not silently re-bound to the builtin
`hline(...)`	Statement-form + numeric price → **nopython** constant series (`hline`, `hline_2`, ...) + synthesized `__drawings`. Assigned / non-numeric handles → object mode
`fill(..., title=…)`	Statement-form → **nopython** all-`nan` series key (band color in `plot_meta` / drawings). Expression-form / mixed object scripts emit `__drawings` in-loop

Compile-path `request.security` never invents foreign OHLCV from chart close. Foreign tickers and complex expressions (UDFs, `ta.*` inside the third arg, multi-value constructs beyond simple chart series) lower to `np.nan`. Use interpret + a real `data_feed` / `data_provider` for multi-asset work.

Internals

Path	Role
`src/pynescript/compiler/compiler.py`	`CompilerVisitor`, emit numeric/object, security/name/fill/hline lowering
`src/pynescript/compiler/numba_builtins.py`	JIT kernels + object-mode `safe_*`
`src/pynescript/compiler/strategy_broker.py`	Compile broker
`src/pynescript/compiler/engine.py`	Façade, caches, `CompiledScript.run`, Numba cache recovery
`src/pynescript/runtime/host.py`	Package SoT `Runtime._run_compiled` envelope; packs OHLCV + `time=`
`backend/runtime.py`	Compat re-export of `pynescript.runtime.host`
`scripts/compare_interp_compile.py`	Interpret↔compile series parity harness
`tests/test_compiler_numba.py`, `test_compiler_objects.py`, `test_compiler_strategy.py`	Coverage
`docs/COMPILER_PLAN.md`	Design history and remaining work (repo, not hosted)

Data layout

Unlike interpreter `PineSeries` deques:

```
open_arr, high_arr, low_arr, close_arr, vol_arr, time_arr
user_arr = np.full(n_bars, np.nan)                # or dtype=object for UDT
plot_i   = np.full(n_bars, np.nan)
for __bar_idx in range(n_bars):
    user_arr[__bar_idx] = …
    plot_i[__bar_idx] = …
```

- Bare `time` / `time_close` / `last_bar_time` / calendar extractors read `time_arr` (synthetic when omitted).
- `var` lowers to “init only when still na / first write” patterns so carry matches Pine without declaration sets.
- Numeric mode returns a **tuple** of plot arrays (host maps titles); object mode returns a **dict** (plots + `__drawings` / strategy extras).

Caches and corrupt-Numba recovery

Three layers:

1. **In-process source LRU** (sha256 of raw and/or sanitized source) + secondary IR cache keyed by generated-code hash.
2. **Disk IR** (default on): modules under `PYNE_COMPILE_CACHE_DIR` or `$XDG_CACHE_HOME/pynescript/compile`. The source→IR index JSON carries `"v": 9` (`engine._DISK_META_VERSION`). Bump that integer when generated IR semantics change so stale modules are ignored (source hash alone is stable across emitter fixes).
3. **Numba function cache** (`.nbi` / `.nbc`) next to `numba_builtins` and under disk-IR `__pycache__/`.

Truncated/corrupt Numba pickle files raise `EOFError` / `pickle.UnpicklingError` on load. The engine wraps execute/warm paths with **purge + single retry** via `clear_numba_function_caches` (does not fail the script for a bad cache alone). Manual clean slate:

```
from pynescript.compiler import (
    clear_compile_cache,
    clear_disk_compile_cache,
    clear_numba_function_caches,
)
clear_compile_cache()
clear_disk_compile_cache()
clear_numba_function_caches()
# shell: rm -rf ~/.cache/pynescript/compile \
#   src/pynescript/compiler/__pycache__/numba_builtins*.nb*
```

Clear disk + Numba caches after compiler emitter or kernel edits if results look stale.

Invariants & edge cases

1. **Same AST front-end.** No second parser—compile bugs are lowering bugs.
2. **OHLCV (+ time when provided) length equality** enforced in `CompiledScript.run`.
3. **Warm-up** ignores exceptions on a dummy 16-bar series (JIT or first-run); non-nopython failures may surface on the first real run.
4. **Result normalization** converts Numba typed maps to plain dicts; `__drawings` / `__events` stay Python lists.
5. **User series shadow bare builtins** in `visit_Name` (`ad`, `tr`, `n`, ...) once `*_arr` is allocated.
6. **`request.security` is not multi-asset on compile.** Only chart-symbol simple OHLCV passthrough; everything else is `na`.
7. **Interpret still owns the full surface.** Exotic builtins, real multi-symbol feeds, and rich strategy metrics remain interpret-first until lowered.

Worked example

Pine:

```
//@version=6
indicator("c")
s = ta.sma(close, 14)
plot(s, "sma")
```

Generated shape (illustrative; numeric mode returns a tuple of series):

```
@numba.njit(cache=False)
def execute_script_compiled(open_arr, high_arr, low_arr, close_arr, vol_arr, time_arr):
    n_bars = len(close_arr)
    s_arr = np.full(n_bars, np.nan)
    plot_0 = np.full(n_bars, np.nan)
    for __bar_idx in range(n_bars):
        s_arr[__bar_idx] = numba_sma(close_arr, 14, __bar_idx)
        plot_0[__bar_idx] = s_arr[__bar_idx]
    return (plot_0,)
# Host packs titles → {'sma': plot_0}
```

Interpret ↔ compile parity

Full contract (tolerances, buckets, intentional diffs): Interpret ↔ compile parity.

```
# From repo root — nan-aware allclose on common series keys
python scripts/compare_interp_compile.py --bars 1000 --limit 50
python scripts/compare_interp_compile.py --glob 'ta_*.pine' --bars 200
python scripts/compare_interp_compile.py --files tests/fixtures/parity/pine/strategy_01_entry_long.
python scripts/compare_interp_compile.py --ignore-hline-keys --ignore-fill-keys --strict-keys
```

Writes `.cache/interp_compile_parity.json`. First-party `hline` / `fill` / `bgcolor` / `plotshape` / `barcolor` / `plotarrow` / `plotbar` / `plotcandle` keys match interpret ↔ compile (0.3.12); `drawings` is geometry-only. Ignore flags are for leftover corpus noise.

Failure modes

Symptom	Cause
`numba is required for numeric compile mode` (`CompileNumbaRequiredError`)	Install numba **or** force/land in object mode (drawings/UDT/strategy already do)
Empty generated code	Visitor returned blank—unsupported top-level shape (`CompileEmitError`)
Missing `execute_script_compiled`	Emit bug or failed `exec` (`CompileLoadError`)
`EOFError` / `UnpicklingError` during JIT load	Corrupt Numba `.nb*` —engine purges and retries; if stuck, call `clear_numba_function_caches`
Silent semantic drift	Kernel seed differences—run `scripts/compare_interp_compile.py`
All-`nan` from `request.security`	Foreign symbol or non-simple expression on compile path (by design)
Wrong values for `ad` / `tr` after assignment	Fixed by user-series shadowing; if stale, clear compile/Numba caches
Strategy metrics missing	Use interpret or inspect `__events` / broker fields only
Calendar/`time` off by hours	Host omitted `time=`; synthetic 1m bar opens used

Remaining work (summary)

Expand Numba `ta.*` surface, richer drawings, nested UDT methods, broader `request.*` (still interpret-first for multi-asset; `mode=auto` already skips any `request.` source), and grow systematic interpret↔compile parity coverage per lowered construct. In-process + disk IR cache (`_DISK_META_VERSION = 9`) and product prewarm are landed. See repo `docs/COMPILER_PLAN.md` for the living checklist.

See also

- Numba path
- Strategy broker
- Interpret ↔ compile parity
- Runtime hub
- request.* / input.* (interpret)
- Numerical validation

NUMBA

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Numba path" description: "JIT bar loops, numba_builtins kernels, NA-as-NaN, and performance characteristics."

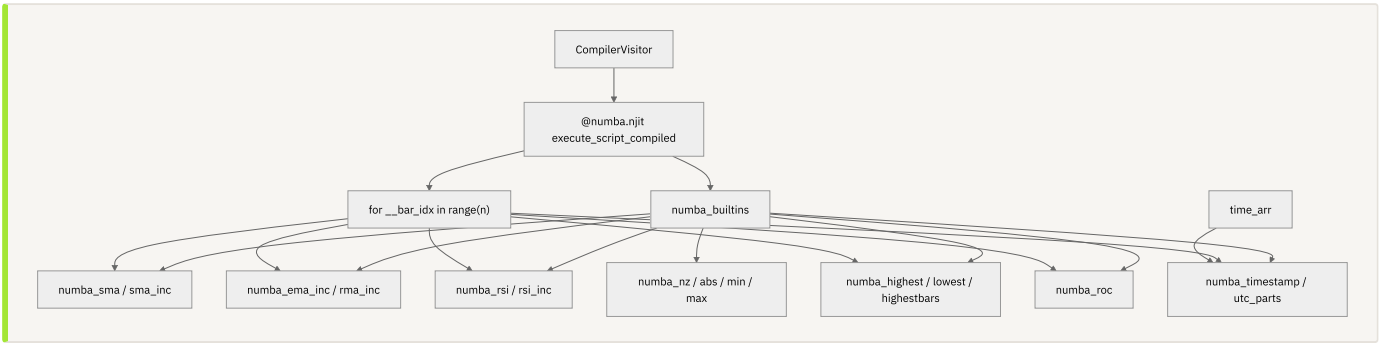
NUMBA PATH

Abstract

Numeric compile mode emits a single `@numba.njit` function whose body is the script's bar loop. Pine builtins that cannot cross the JIT boundary are replaced by **hand-written kernels** in `numba_builtins.py` that take full arrays plus the current index (or a small float state vector for `*_inc` forms). The result is large speedups on multi-thousand-bar series after a one-time JIT warm-up—at the cost of a restricted language subset and `np.nan` as the missing-value sentinel.

Kernel contracts below match interpret `ta.*` / calendar helpers where parity was fixed; intentional deviations should be documented on the kernel docstring and here.

Conceptual model



Interface surface

Kernels are defined with `@numba.njit(cache=True)` in `numba_builtins.py`. Generated entry points use `@numba.njit(cache=False)` in the in-memory IR string; when the engine writes a **disk IR** module it rewrites that decorator to `cache=True` so Numba can store machine code next to a real file path (see Cache).

Generated code imports `from pynescript.compiler.numba_builtins import *` so call sites are bare names. Prefer `*_inc` when the visitor allocates fixed state (`__ema0_st` , ...).

Kernel table (interpret-parity highlights)

Kernel	Semantics
`numba_sma(arr, period, i)`	Mean of last `period` bars ending at `i`; `nan` if warm-up or any window `nan`
`numba_ema` / `numba_ema_inc`	SMA seed, then EMA (`α = 2/(period+1)`). **Inc seed is NaN-safe**: while the accumulator is still `nan` and `j >= period-1`, try SMA over `arr[j-period+1 : j+1]` only when every sample is finite. Nested EMA / DEMA no longer stuck all-NaN after leading NaNs
`numba_rma_inc(arr, period, i, st)`	Wilder RMA with the **same NaN-window-safe SMA seed** as `numba_ema_inc`. After seed, NaN inputs hold the previous RMA. Required for `rma(tr)` / expanded ADX so compile is not all-NaN or ~0 after warm-up
`numba_rsi` / `numba_rsi_inc`	**Wilder RSI**, not rolling simple-window RSI. SMA of first `period` deltas (bars `1..period`), then RMA of gain/loss (`α = 1/period`). First valid bar is **`i == period`** (`period` deltas need `period+1` prices). `avg_loss == 0` → `100.0`
`numba_highest` / `numba_lowest`	Window extrema; full window required (`i >= period-1`); NaN samples skipped
`numba_highestbars` / `numba_lowestbars` (+ `_inc`)	TV **negative** bars-back offsets: `0` if current is extreme, `-1` one bar ago, ..., `-(length-1)`. Returns **`-1.0`** when window not full, invalid length, or all-NaN. **Oldest** extreme wins ties (strict `>` / `<`)
`numba_roc(arr, length, i)`	`100 * (src - src[length]) / src[length]` at bar `i`; `nan` if `i < length`, baseline 0/NaN, or src NaN
`numba_timestamp(y, m, d, h=0, mi=0, s=0)`	UTC epoch **ms** from calendar components; month/day overflow normalized (e.g. `month=0` → prior Dec) for TTM-style windows
`numba_utc_parts(ms)`	`(year, month, dayofmonth, hour, minute, second, dayofweek)` floats; Pine `dayofweek` is `1=Sun … 7=Sat`
`numba_nz(val, replacement)`	Replace `nan`
`numba_abs` / `numba_min` / `numba_max`	Scalar helpers

Full inventory lives in `src/pynescript/compiler/numba_builtins.py` (njit batch + `*_inc` + pure-Python object-mode helpers).

Calendar / `time_arr`

Bar open time is the host `time_arr` series (synthetic when the runner omits timestamps).

Pine form	Lowering
bare `year` / `month` / `dayofmonth` / `hour` / `minute` / `second` / `dayofweek`	`numba_utc_parts(time_arr[__bar_idx])[idx]`
`year(t)` ... (optional time arg)	`numba_utc_parts(src)[idx]` with `src` defaulting to `time_arr[__bar_idx]`
`timestamp(y, m, d[, h, mi, s])`	`numba_timestamp(...)` (leading timezone string args dropped → UTC stub)
`time` / `time_close` (approx)	`time_arr[__bar_idx]` / `+ 59999` ms

History access

`close[n]` lowers to array indexing at `__bar_idx - n` with bounds → `nan` (parallel to interpreter `None`, different sentinel).

Control flow

`if` / `for` / `while` / `once` emit Python control flow **inside** the njit function—supported Numba subset only (no heap objects, no Python lists of mixed types). `once` lowers to a persistent `__once_fired_N` bool set when the body runs (compile is historical: `barstate.isconfirmed` is always true). A `once` inside a UDF forces object mode so the flag is module-level.

Cache

Three layers interact:

Layer	Location / key	Role
Source LRU	in-process, sha256 of source	`CompiledScript` reuse
IR LRU	in-process, sha256 of generated Python	share warm callable across comment-only variants
Disk IR	`PYNE_COMPILE_CACHE_DIR` or `$XDG_CACHE_HOME/pynescript/compile`	persist generated modules; disable with `PYNE_COMPILE_DISK_CACHE=0`. Index JSON `"v"` must equal `engine._DISK_META_VERSION` (**9**)

Numba function cache (`.nbi` / `.nbc`):

- Hand-written kernels: `@njit(cache=True)` → artifacts under `…/pynescript/compiler/__pycache__/`.
- Generated entry: IR is emitted with `cache=False` (safe for `exec` from `<string>`). Disk write rewrites `@numba.njit(cache=False)` → `cache=True` so Numba's file locator can cache machine code under the disk IR `__pycache__/`.
- Truncated/corrupt pickle loads raise `EOFError` / `UnpicklingError`. The engine purges known `.nbi` / `.nbc` via `clear_numba_function_caches()` and recompiles once instead of failing the script.

```
from pynescript.compiler import clear_numba_function_caches, clear_disk_compile_cache
clear_numba_function_caches()
clear_disk_compile_cache()
```

Internals

Path	Role
`src/pynescript/compiler/numba_builtins.py`	Kernels (`cache=True`)
`src/pynescript/compiler/compiler.py`	`_emit_numeric_mode`, call lowering, `time_arr` calendar
`src/pynescript/compiler/engine.py`	Disk IR rewrite, Numba cache recovery, prewarm
`tests/test_compiler_numba.py`	Correctness / parity

Expanding coverage is primarily **new kernels + visitor call mapping**, not changes to the bar-loop skeleton.

Invariants & edge cases

1. **No Python objects in numeric mode.** Strings, UDTs, maps, drawings force object mode before njit is applied.
2. **Warm-up cost.** First `compile_script` invokes a dummy run; production servers should cache `CompiledScript` and prewarm (`PYNE_COMPILE_PREWARM` default **on**, `POST /compile/prewarm`).
3. **EMA/RMA seed** uses sliding all-finite SMA windows—not "seed once at bar `period-1` only if that fixed slice is clean." Leading NaN on `tr` / nested EMA sources must eventually produce values.
4. **RSI is Wilder**, first valid at `i == period` . Do not compare against a simple N-bar average-gain RSI.
5. **highestbars/lowestbars** return negative offsets and `-1` on incomplete windows; ties keep the oldest bar.
6. **Disk entry cache** requires the rewrite to a real file path; pure `exec` of `cache=False` IR does not populate Numba's on-disk function cache for the entry function.
7. **Anaconda static libpython.** Packaging notes in AGENTS/build docs apply when shipping JIT-heavy binaries.

Worked examples

Benchmark mental model

```
interpret:  O(bars × ast_nodes × python_dispatch)
numeric:    O(bars × lowered_ops) in machine code after JIT
```

For simple SMA scripts, internal notes cite order-of-magnitude speedups versus list-based interpretation on long series (see repo `docs/COMPILER_PLAN.md` qualitative claims).

Using from Pro API

```
Runtime(...).run(source, ohlcv, mode="compile")
```

Converts bar dicts → arrays (including `time=` bar-open ms), compiles, reshapes plots into the standard envelope, flags `object_mode` when applicable. `generated_code` is omitted unless `PYNESCRIPT_RETURN_GENERATED_CODE=1` .

Failure modes

Symptom	Cause
TypingError from Numba	Unsupported construct leaked into numeric emit
All-`nan` plots	Period longer than series; history index bugs; or (fixed) NaN-poisoned EMA/RMA seed on nested sources
Nested DEMA / `rma(tr)` all-NaN	Pre-fix seed; current kernels sliding-window all-finite SMA seed
RSI diverges vs interpret	Expect Wilder seed/RMA, not rolling window RSI
`highestbars` sign / magnitude off	TV negative offsets; incomplete window → `-1`
`EOFError` / UnpicklingError at load	Corrupt `.nbi` / `.nbc` — engine purges and retries; manual `clear_numba_function_caches()`
ImportError numba	Environment missing dependency
Object mode unexpectedly	Visitor detected drawing/UDT/map—inspect `visitor.object_mode`

Performance notes

- Prefer **one compile, many runs** with different OHLCV.
- Keep scripts in the numeric subset for realtime ticks.
- Host prewarm of `numba_builtins` + common scripts shifts cold JIT off the first user request.
- Object mode still wins over AST walking for UDT-heavy scripts but will not match peak njit throughput.

See also

- Compiler overview
- Technical builtins (interpret)
- Series & history

STRATEGY BROKER

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

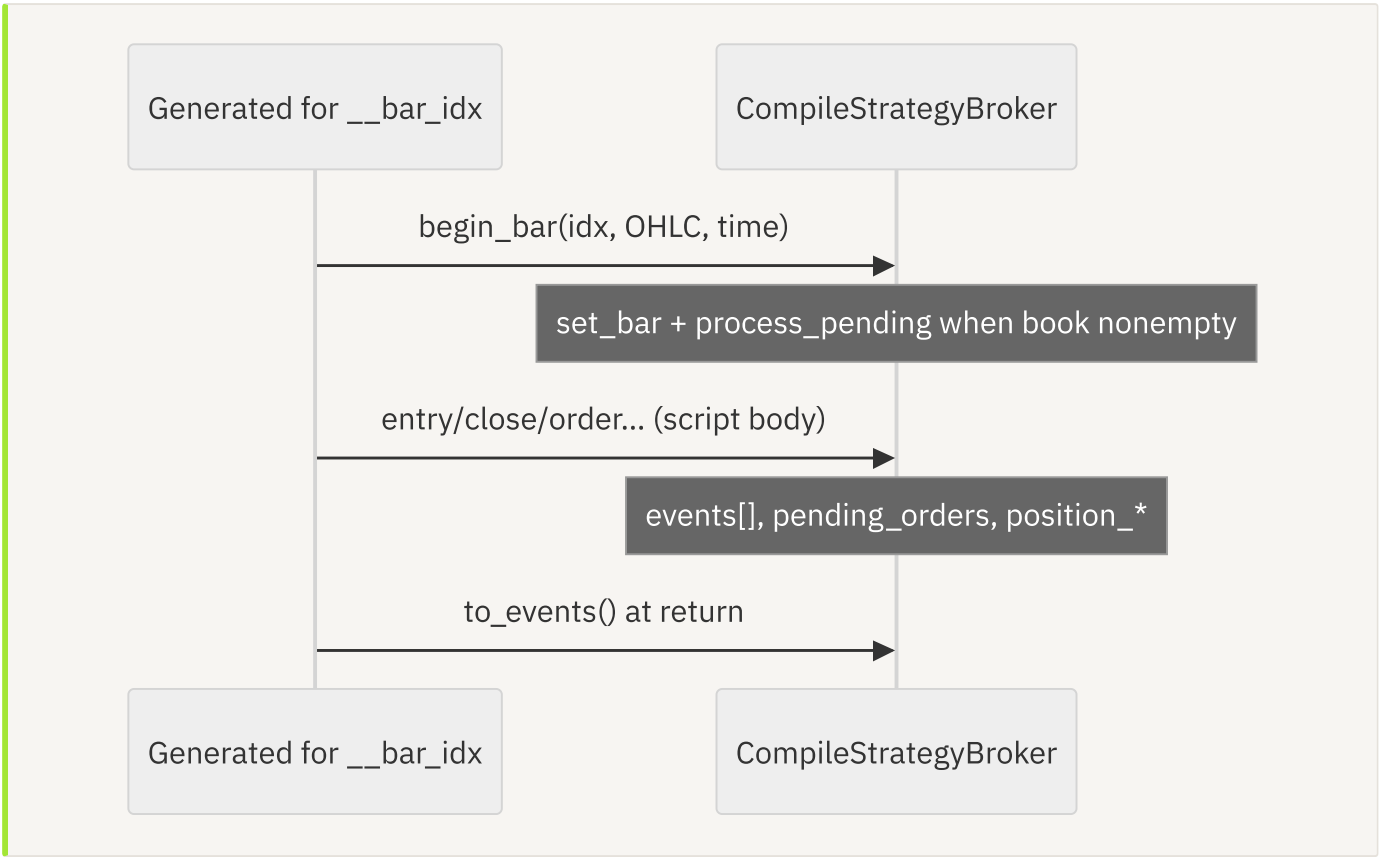
title: "Compile strategy broker" description: "CompileStrategyBroker: pending orders, OHLC fills, OCA, commission, and event export."

COMPILE STRATEGY BROKER

Abstract

When `CompilerVisitor` detects strategy usage, object-mode emission instantiates **`CompileStrategyBroker`**—a lightweight broker aligned with the interpreter's fill-before-script cadence, but designed for generated loops (no AST, no full `StrategyState` metric surface). It supports market entry/close and pending limit/stop/stop-limit orders with per-bar OHLC triggers, OCA cancel/reduce, commission, and slippage.

Conceptual model



Interface surface

Construction (from `strategy()` kwargs when lowered)

```
CompileStrategyBroker(
    initial_capital=100_000.0,
    commission_value=0.0,
    commission_type="percent",  # percent | cash_per_order | cash_per_contract
    slippage_ticks=0,
    mintick=0.01,
    pyramiding=0,
    default_qty_type="fixed",
    default_qty_value=1.0,
    avg_price_model="stock",  # stock | futures | inverse (pynescript extension)
    leverage=1.0,             # futures UI: buying power / margin divisor
)
```

`avg_price_model` is lowered from `strategy(..., avg_price_model=...)`. The compile broker is a **single net lot**: partial reduce already keeps `position_avg_price` sticky (futures-like). Multi-leg stock reweight on partial close is interpret-first until the compile path grows an open-trade list.

`leverage` scales percent/cash default qty (`margin × leverage / price`) and free `cash` (equity – notional/leverage).

Bar context

```
# Preferred generated hot path (one call; skips pending walk when empty)
broker.begin_bar(bar_index, open_, high, low, close, bar_time=…)
# Equivalent two-step still exists:
broker.set_bar(bar_index, bar_time, mark, open_=…, high=…, low=…, close=…)
broker.process_pending_orders(open_=…, high=…, low=…, close=…)
```

Orders

Method	Role
`entry(...)`	Market or pending entry; reverse flattens opposite
`close` / `close_all`	Reduce/flat with optional price
Pending book	`PendingOrder` with type, direction, qty, limit/stop, OCA, `max_fill_per_bar`, `is_entry`

Fill logic (`_trigger_price`)

Type	Long trigger	Short trigger	Fill price idea
market	—	—	close (immediate path)
limit	`low <= limit`	`high >= limit`	gap-aware vs open
stop	`high >= stop`	`low <= stop`	gap-aware vs open
stop-limit	stop traded and limit available	symmetric	limit price

Partial fills: `max_fill_per_bar` caps quantity per bar; residual stays pending.

OCA

After a fill, siblings in the same `oca_name`:

- **`cancel`** — remove sibling, emit `cancel`
- **`reduce`** — decrease sibling qty by fill size
- **`none`** — no interaction

Economics

- **Slippage:** `± slippage_ticks * mintick` by direction on fills.
- **Commission:** percent of notional, cash per order, or cash per contract.
- **Position:** signed `position_size`, `position_avg_price`, `netprofit`, `equity` helpers for return payload.

Events

`_emit` appends dicts with keys compatible with strategy event serialization (`kind`, `id`, `direction`, `qty`, `order_type`, `limit`, `stop`, `oca_name`, `comment`, `bar_index`, `bar_time`, `ohlc`). Object-mode return includes `'__events': __strategy.to_events()` plus position/netprofit/equity snapshots.

Internals

Path	Role
`src/pynescript/compiler/strategy_broker.py`	Broker implementation
`src/pynescript/compiler/compiler.py`	Emits `__strategy` wiring in object mode
`src/pynescript/ast/evaluator/builtins/strategy.py`	Interpreter counterpart (richer)
`tests/test_compiler_strategy.py`	Compile strategy coverage

NA helpers treat `None`, blank/`na` strings, and float NaN as missing prices when classifying order types.

Invariants & edge cases

1. **Same order as interpreter:** `begin_bar` (or `set_bar` → `process_pending_orders`) → script body.
2. **Closed-trade list is partial.** The compile broker now keeps `closed_trade_records` and some `closedtrades_*` accessors (profit, comments, max drawdown/runup). Prefer interpret for the full `strategy.closedtrades.*` / `opentrades.*` metric surface.
3. **Direction normalization** accepts `strategy.long`, `long`, `1`, `buy`, etc.
4. **Reverse on opposite entry** via `close_all(comment="reverse")` then open.
5. **Event dicts are mutable lists**—fine for run output; not frozen dataclasses.

Worked examples

Generated prologue (illustrative)

```
__strategy = CompileStrategyBroker(initial_capital=100000.0)
for __bar_idx in range(n_bars):
    __strategy.begin_bar(__bar_idx,
        float(open_arr[__bar_idx]), float(high_arr[__bar_idx]),
        float(low_arr[__bar_idx]), float(close_arr[__bar_idx]),
        bar_time=int(time_arr[__bar_idx]))
    # lowered strategy.entry / close calls
return {..., '__events': __strategy.to_events(),
        '__position_size': __strategy.position_size,
        '__netprofit': __strategy.netprofit,
        '__equity': __strategy.equity}
```

Limit buy pending

```
strategy.entry("L", strategy.long, limit=low - syminfo.mintick)
```

Object-mode lowers to a pending limit; subsequent bars fill when `low` trades through.

Failure modes

Symptom	Cause
No fills	Triggers never met; or market path not invoked
OCA sibling remains	`oca_type` none / name mismatch
PnL vs interpret drift	Commission/slippage/mintick defaults differ
Missing events in API envelope	Host only maps plots—not `__events`

See also

- Strategy builtins (interpret)
- Events
- Compiler overview
- Numba path

PARITY

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Interpret ↔ compile parity" description: "Plot-series parity between Runtime interpret and compile modes: harness, tolerances, and known sentinel differences."

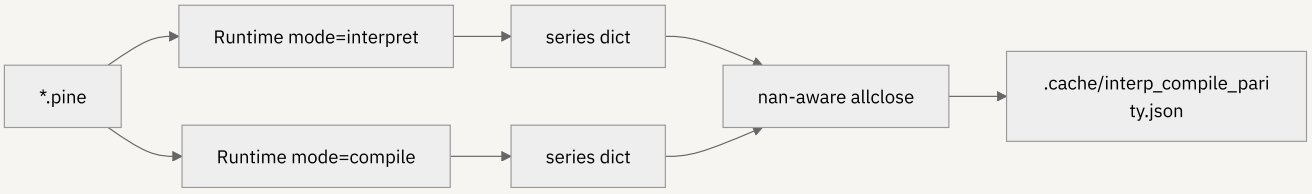
INTERPRET ↔ COMPILE PARITY

Abstract

PYNE ships two bar engines that should agree on **numeric plot series** for scripts in the compile surface: the AST interpreter and the generated Numba/object-mode loop. This page is the short contract for that agreement—how to measure it, what “equal” means, and where intentional differences live (`None` vs `nan` , host `time_arr` , structural keys).

Strategy **event** parity (Python vs TypeScript worker) is a separate oracle (`tests/test_parity.py` + fixtures). Here the focus is **plot values** on one host: `pynescript.runtime.Runtime` .

Conceptual model



Both modes receive the **same** synthetic (or host) OHLCV. Compile additionally builds `time_arr` from bar `time` fields so `time` / calendar expressions line up with the interpret `time` series.

Interface surface

Corpus harness

```
# From repo root — first-party fixtures by default
python scripts/compare_interp_compile.py --bars 1000 --limit 50
python scripts/compare_interp_compile.py --files tests/fixtures/parity/pine/strategy_01_entry_long.
```

Flag / output	Role
Default inputs	`tests/fixtures/parity/pine/*.pine` (first-party)
Tolerances	`rtol=1e-5`, `atol=1e-6` (nan-aware)
Report	`.cache/interp_compile_parity.json`
Exit 0	No value/ `nan` mismatches on common series keys

Buckets: `OK`, `fill_background_only`, `both_error_same`, `expected_error`, `both_error`, `MISMATCH`, `interp_error`, `compile_error`, `structural_only`. Matched errors on both backends (`both_error_same`) and intentional demo failures (`expected_error`, e.g. auto-fib / pivot-depth) count as success unless `--strict-errors`. First-party `hline` / `fill` / `bgcolor` / `plotshape` / `barcolor` / `plotarrow` / `plotbar` / `plotcandle` keys match interpret ↔ compile (0.3.12); `drawings` is geometry-only. `--ignore-hline-keys` / `--ignore-fill-keys` remain optional for leftover **corpus** key noise; `--strict-keys` fails on any one-sided key.

Focused unit tests

Test	Scope
`tests/test_dividend_yield_parity.py`	Host-series copy-on-assign + time history (interpret/compile)
`tests/test_compiler_numba.py`	Kernel-level numeric correctness
`tests/test_parity.py`	Strategy **event** fixtures (not plot allclose)

Product path

Pro API `/run` defaults to `mode=auto`: compile when `_compile_eligible` passes (no top-level `import`, no `request.` token), otherwise interpret with `compile_fallback_reason`. Parity harness always forces explicit `interpret` vs `compile` so fallbacks do not hide drift.

Official TradingView® builtins (0.6.2): interpret UDF series-parameter history (`s[1]` per call site) closed the remaining P1p plot MISMATCH tail on that set — **0 MISMATCH** vs compile. Optional set0x community corpus is unmeasured, not a missing language surface.

What must match

1. **Common plot keys** — float series after `None` ↔ `nan` normalization.
2. **Warm-up `na`** — leading missing samples align (interpret `None` / compile `nan`).
3. **`time` / bar-open ms** — when OHLCV includes `time`, or both use the synthetic `i * 60_000` fallback.
4. **Structured errors** — scripts that `runtime.error` (e.g. insufficient pivots) should fail on both with messages that normalize equal (`both_error_same`).

Intentional / soft differences

Topic	Policy
Sentinels	Interpret: `None` ; Numba: `np.nan` . Compare with nan-aware equality, not `==` .
Hline / fill / bgcolor keys	May appear on one mode only; ignore with harness flags unless you need strict key sets.
Unresolved imports / stubs	Non-numeric plot cells serialize as `na` (null)—not string stubs.
Outside compile surface	UDT-heavy / full `request.*` / rich strategy analytics stay interpret-first; `auto` falls back.
Disk / Numba caches	Stale IR after kernel edits can fake mismatches—clear compile + Numba caches (see harness docstring).

Internals

Path	Role
`scripts/compare_interp_compile.py`	End-to-end series compare
`tests/test_interp_compile_parity.py`	Always-on smoke + harness flag tests
`src/pynescript/runtime/host.py`	`_run_compiled` , OHLCV + `time_arr` packing
`backend/runtime.py`	Compat re-export of the package host
`src/pynescript/compiler/engine.py`	Compile façade + result normalize
`src/pynescript/compiler/numba_builtins.py`	JIT kernels under test

Worked example

```
python scripts/compare_interp_compile.py \
  --files tests/fixtures/parity/pine/strategy_01_entry_long.pine \
  --bars 500
# → OK  (or MISMATCH with first differing bar/key in the JSON report)
```

Failure modes

Symptom	Likely cause
Systematic offset after warm-up	EMA/RSI seed or window clamp drift in a kernel
Only-compile empty plots	Script still object-mode / plot not lowered; or cache corruption
`time` diverges	Host omitted times on one path only; check `time_arr` packing
Structural-only keys	hline/fill/background naming — use ignore flags or align collectors

See also

- Compiler overview
- Numba path
- Series & history
- Runtime hub
- Numerical validation
- PyneTS parity — TypeScript vs Python plots